

Texture Explorer

A literate program about texture maps, bumps and normals
written in C++20, HLSL and Direct3D 11

October 7, 2026

Contents

1	Introduction	2
7	Parametric shapes	4
32	Materials and their maps	11
56	The probe: from a texel to a normal	19
65	The shaders	22
83	The renderer	27
117	The application and its user interface	37
175	The main program	52
189	Building the program	56
193	Exercises	58
	Names of the modules	59

How to read this document. The program is presented in numbered sections, each with an informal commentary and (usually) a piece of code. A piece of code may refer to a *module* by name, written $\langle \textit{Name} \ n \rangle$; the number n is the section where that module is defined, and the module’s text belongs where its name appears. “+≡” marks a continuation of a module defined earlier. Modules named $\langle \text{File} \ \textit{path} \ n \rangle$ are complete files: running `lp.ps1 tangle` produces them, and `lp.ps1 check` confirms that they are identical to the files of the project. Keywords are in **bold type**, comments in *italic*, and the sign $\hookrightarrow$ continues a long line.

1. Introduction.

This document is the complete program *Texture Explorer*, a small interactive tool for a course in computer graphics. It shows a material—a set of images that together describe a surface—and lets the student look at every image of the set, apply the material to a simple solid, move the camera and the light, and, above all, *ask questions*: pointing with the mouse at any texel of any map, the program answers with the texture coordinates of that texel, the values stored there in every map, and the normal vector that the bump map produces at that point, both in the tangent space of the surface and in the coordinate system of the world.

The program is written in the style that Donald E. Knuth called *literate programming*: instead of writing a program for a computer and then documenting it, we write an essay for human beings in which the program appears, piece by piece, in the order in which it is easiest to understand. The pieces are later rearranged mechanically into the files that a compiler wants. The tool that does this, `lp.ps1`, is described in the file `README.md` next to this web.

2. The main ideas a student should take away are few, and they reappear constantly:

- *Texture space*. Every point of a surface carries coordinates (u, v) in the unit square; the images of the material are indexed by them. How a shape is mapped onto the square is a design decision, and different shapes show different trade-offs.
- *Height fields*. A grey image $h(u, v)$ can be used in two very different ways: as a *bump map*, which only tilts the normal used for shading, or as a *displacement map*, which really moves the surface.
- *Tangent space*. The normal computed from a bump map is expressed relative to the surface, in the frame $(\mathbf{T}, \mathbf{B}, \mathbf{N})$; it becomes a vector of the world only after multiplication by that frame.
- *Roughness*. A roughness map changes, point by point, how sharp or blurry the specular highlight is, through a microfacet model of reflection.

3. The program consists of the following files, each of which is a module of this web.

<code>src/Mesh.h, src/Mesh.cpp</code>	parametric shapes and their texture mapping
<code>src/Material.h, src/Material.cpp</code>	loading the maps of a material
<code>src/Inspector.h, src/Inspector.cpp</code>	the probe: values and normals at a texel
<code>shaders/pbr.hlsl</code>	displacement, bump mapping and lighting on the GPU
<code>shaders/gizmo.hlsl</code>	coloured lines: arrows and the light marker
<code>src/Renderer.h, src/Renderer.cpp</code>	Direct3D 11 device, targets and drawing
<code>src/App.h, src/App.cpp</code>	the user interface
<code>src/main.cpp</code>	the window and the main loop
<code>CMakeLists.txt</code>	the build description

We shall meet them roughly from the bottom up: first the geometry, because texture coordinates are a property of geometry; then the images; then the mathematics of the probe; then the GPU code that does the same mathematics for every pixel; and finally the machinery around them. The images themselves come from two libraries of free (CC0) materials, *ambientCG* and *Poly Haven*; a PowerShell script downloads ten of them and writes a *manifest* that describes their origin.

4. One frame of the program, seen from far away, goes like this. The user interface is rebuilt from scratch (Dear ImGui is an “immediate mode” library); while it is being built we learn whether the mouse is over one of the images, and where. If it is, the *inspector* computes everything that can be said about that texel. Then the 3D scene is drawn into an off-screen image, together with arrows that show the inspector’s vectors at the corresponding point of the solid, and the off-screen image is shown in the “Viewport” panel. Finally the whole user interface is drawn on the screen.

5. *Conventions*. Throughout the program the world is right-handed with the y axis pointing up, and the camera and the light orbit around the origin, where the solid sits. Texture space follows the convention of Direct3D: $(0, 0)$ is the *top-left* corner of an image, u grows to the right and v grows *downwards*.

This has a consequence that is worth stating once and for all. Let $\mathbf{P}(u, v)$ be a point of a surface, $\mathbf{T} = \partial\mathbf{P}/\partial u$ and $\mathbf{B} = \partial\mathbf{P}/\partial v$ its tangent and bitangent, and $\mathbf{N}$ the outward normal. An observer outside the surface sees $\mathbf{T}$ pointing to the right and, because v grows downwards in the image, $\mathbf{B}$ pointing *down*. Right $\times$ down is *into* the surface in a right-handed system, so for a correctly oriented (not mirrored) texture

$$\mathbf{T} \times \mathbf{B} = -\mathbf{N}.$$

Every shape in the next part has been checked against this rule.

6. Vectors of unit length can be stored as colours by the affine map $\mathbf{n} \mapsto \frac{1}{2}\mathbf{n} + \frac{1}{2}$, which takes $[-1, 1]$ to $[0, 1]$. Normal maps use it, and so do the colour swatches of the inspector and the “normal” views of the shader. A normal that points straight out of the surface, $(0, 0, 1)$ in tangent space, becomes the characteristic lavender $(0.5, 0.5, 1)$ of normal maps.

7. Parametric shapes.

A texture is applied to a surface by giving every point of the surface a pair of texture coordinates. For the simple solids of this program—a sphere, a pyramid, a cylinder and a capsule—the cleanest way to do that is to describe the surface itself as a function of two parameters. We call such a function a *patch*: it maps the unit square of parameters (a, b) to a point of the surface, and it reports, besides the position, the outward normal, the two partial derivatives $\partial \mathbf{P} / \partial u$ and $\partial \mathbf{P} / \partial v$, and the texture coordinates (u, v) of the point.

A shape is a list of patches. The sphere needs only one; the cylinder needs three (the side and two caps); the pyramid needs five.

The same patch functions are used twice, and this is the key to the whole design. The renderer samples them on a fine grid to build the triangle meshes that the GPU draws; the inspector *inverts* them to answer the question “where on the solid is texel (u, v) ?”. Because both use the same functions, the arrows drawn by the inspector land exactly on the pixels that show the probed texel.

```
< File src/Mesh.h 7 > ≡
#pragma once
// Parametric shapes. Every shape is a set of patches; each patch maps a parameter
// square (a,b) in [0,1]^2 to a surface point with position, normal, tangent frame
// and texture coordinates. The same functions build the GPU meshes and answer the
// inspector's question "where on the object is texel (u,v)?".
#include <DirectXMath.h>
#include <functional>
#include <vector>
#include <cstdint>

< The four shapes 8 >

< Surface points 9 >

< Vertex layout 10 >

< Patches 11 >

< Mesh data 12 >

< Interface of the mesh module 13 >
```

This code is written to the file `src/Mesh.h`.

8. The shapes are numbered so that a combo box can list them; `Count` is not a shape but the number of them.

```
< The four shapes 8 > ≡
enum class Shape { Sphere, Pyramid, Cylinder, Capsule, Count };
const char* ShapeName(Shape s);
```

This code is used in section 7.

9. A `SurfacePoint` is what a patch returns. Note that the derivatives are *not* normalized: their lengths tell how much the surface stretches the texture, which is interesting information even if this program only uses their directions.

```
< Surface points 9 > ≡
struct SurfacePoint {
    DirectX::XMFLOAT3 pos;           // object space
    DirectX::XMFLOAT3 normal;        // unit outward geometric normal
    DirectX::XMFLOAT3 dPdu;          // direction of increasing u (tangent T, not normalized)
    DirectX::XMFLOAT3 dPdv;          // direction of increasing v (bitangent B, not normalized)
    DirectX::XMFLOAT2 uv;
};
```

This code is used in section 7.

10. The GPU receives a more compact description of each vertex. Storing both $\mathbf{T}$ and $\mathbf{B}$ would cost six floats; the usual trick is to store $\mathbf{T}$ (made exactly perpendicular to $\mathbf{N}$) and a single sign w , from which the shader rebuilds $\mathbf{B} = w(\mathbf{N} \times \mathbf{T})$. The sign says whether the texture is mirrored on this part of the surface; with our conventions (§5) it is -1 everywhere.

```
< Vertex layout 10 > ≡
struct Vertex {
    DirectX::XMFLOAT3 pos;
    DirectX::XMFLOAT3 normal;
```

```

    DirectX::XMFLOAT4 tangent; // xyz = T orthogonalized against N, w = sign so B = w *
        ↪ cross(N, T)
    DirectX::XMFLOAT2 uv;
};

```

This code is used in section 7.

11. A patch is a function plus a little bookkeeping. Some patches are *invertible*: their texture coordinates are an affine function of the parameters, $u = u_0 + a(u_1 - u_0)$ and $v = v_0 + b(v_1 - v_0)$, so that from (u, v) we can recover (a, b) and therefore the point. Other patches, like the caps of the cylinder, map the texture by a projection that overlaps another patch; the inspector ignores them.

```

⟨ Patches 11 ⟩ ≡
struct Patch {
    std::function<SurfacePoint(float a, float b)> eval;
    // Patches whose uv equals (lerp(u0,u1,a), lerp(v0,v1,b)) form a one-to-one chart
    // of the texture and can be inverted by the inspector.
    bool invertible = false;
    float u0 = 0, u1 = 1, v0 = 0, v1 = 1;
};

```

This code is used in section 7.

12. A mesh, ready to be copied into GPU buffers, is a list of vertices and a list of indices, three per triangle.

```

⟨ Mesh data 12 ⟩ ≡
struct MeshData {
    std::vector<Vertex> vertices;
    std::vector<uint32_t> indices;
};

```

This code is used in section 7.

13. These four functions are all that the rest of the program needs from this module.

```

⟨ Interface of the mesh module 13 ⟩ ≡
std::vector<Patch> MakePatches(Shape shape);
MeshData BuildMesh(Shape shape, int segments);
// Finds the surface point whose texture coordinate is (u,v). Returns false if no
// invertible patch covers that uv.
bool FindSurfacePoint(Shape shape, float u, float v, SurfacePoint& out);
// Builds the orthonormal tangent frame used by both shader and inspector.
void TangentFrame(const SurfacePoint& p, DirectX::XMFLOAT3& T, DirectX::XMFLOAT3& B,
    ↪ DirectX::XMFLOAT3& N);

```

This code is used in section 7.

14. The implementation file has the usual shape: some private helpers in an anonymous namespace, one function per shape that produces its patches, and then the public functions.

```

⟨ File src/Mesh.cpp 14 ⟩ ≡
#include "Mesh.h"
#include <cmath>

using namespace DirectX;

namespace {
constexpr float kPi = 3.14159265358979f;

⟨ Reminder of the texture-space convention 15 ⟩

⟨ Helper that builds a surface point 16 ⟩

⟨ Patches of the sphere 17 ⟩

⟨ Patches of the cylinder 18 ⟩

⟨ Patches of the capsule 20 ⟩

⟨ Patches of the pyramid 22 ⟩
} // namespace

⟨ Public functions of the mesh module 25 ⟩

```

This code is written to the file `src/Mesh.cpp`.

15. The convention of §5 is repeated in the code, where it is needed.

```
<Reminder of the texture-space convention 15>≡
// Texture-space convention (Direct3D): u grows to the right, v grows DOWN the image.
// For a viewer outside the surface this means cross(dPdu, dPdv) == -N.
This code is used in section 14.
```

16. A one-line helper keeps the patch functions short.

```
<Helper that builds a surface point 16>≡
SurfacePoint Make(XMFLOAT3 p, XMFLOAT3 n, XMFLOAT3 du, XMFLOAT3 dv, float u, float v) {
    return SurfacePoint{p, n, du, dv, {u, v}};
}
```

This code is used in section 14.

17. *The sphere.* The unit sphere in spherical coordinates is $\mathbf{P}(\theta, \varphi) = (\sin \theta \cos \varphi, \cos \theta, -\sin \theta \sin \varphi)$, with the polar angle $\theta = \pi b$ measured from the north pole and the azimuth $\varphi = 2\pi a$. The minus sign in the z component is not a whim: without it the texture would appear mirrored (try it, and check the rule $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$).

On a unit sphere the normal is the position itself. The derivative with respect to φ is $\sin \theta (-\sin \varphi, 0, -\cos \varphi)$; we drop the factor $\sin \theta$, which vanishes at the poles, so that the tangent direction stays defined there. The derivative with respect to θ is $(\cos \theta \cos \varphi, -\sin \theta, -\cos \theta \sin \varphi)$. Texture coordinates are simply $(u, v) = (a, b)$: the whole image is wrapped once around the sphere, stretched horizontally at the equator and pinched at the poles.

```
<Patches of the sphere 17>≡
std::vector<Patch> SpherePatches() {
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float th = kPi * b, ph = 2 * kPi * a;
        const float st = std::sin(th), ct = std::cos(th), sp = std::sin(ph), cp =
            ↪std::cos(ph);
        XMFLOAT3 p{st * cp, ct, -st * sp};
        return Make(p, p, {-sp, 0, -cp}, {ct * cp, -st, -ct * sp}, a, b);
    };
    return {side};
}
```

This code is used in section 14.

18. *The cylinder* has radius $r = 0.7$ and height $H = 1.6$. Its side is the sphere's recipe with a constant radius: the same azimuth, and a height that decreases linearly from $H/2$ to $-H/2$ as v goes from 0 to 1, so $\partial \mathbf{P} / \partial v = (0, -1, 0)$ (up to the factor H).

```
<Patches of the cylinder 18>≡
std::vector<Patch> CylinderPatches() {
    constexpr float r = 0.7f, H = 1.6f;
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float ph = 2 * kPi * a, sp = std::sin(ph), cp = std::cos(ph);
        return Make({r * cp, H / 2 - b * H, -r * sp}, {cp, 0, -sp}, {-sp, 0, -cp}, {0, -1,
            ↪0}, a, b);
    };
    <Caps of the cylinder 19>
    return {side, cap(H / 2, 1), cap(-H / 2, -1)};
}
```

This code is used in section 14.

19. The caps are discs, parametrized in polar form (angle a , radius fraction b), but their texture is applied by a *planar projection*: the disc of radius r is mapped onto the unit square. Each cap therefore shows a round piece of the texture, which is what one expects from a lid, and the caps' texture coordinates overlap those of the side. That is why the caps are not invertible.

The sign ny is $+1$ for the top cap and -1 for the bottom one. Looking at the top cap from above, u must grow along $+x$ and v along $+z$; looking at the bottom cap from below, v must grow along $-z$. The factor $-ny$ in the v coordinate and the derivative $(0, 0, -ny)$ take care of both cases at once. (Check: $\mathbf{x} \times \mathbf{z} = -\mathbf{y} = -\mathbf{N}$ on the top cap.)

```
<Caps of the cylinder 19>≡
// Caps use a planar projection, so their uv overlaps the side's chart.
auto cap = [](float y, float ny) {
```

```

Patch c;
c.eval = [y, ny](float a, float b) {
    const float ph = 2 * kPi * a, x = r * b * std::cos(ph), z = -r * b * std::sin(ph);
    return Make({x, y, z}, {0, ny, 0}, {1, 0, 0}, {0, 0, -ny}, x / (2 * r) + 0.5f, -ny *
        ↪ z / (2 * r) + 0.5f);
};
return c;
};

```

This code is used in section 18.

20. The capsule is a cylinder of radius $r = 0.5$ and half-length $L = 0.5$ closed by two hemispheres. It is a surface of revolution, so it is enough to describe its *profile*: a curve in the (ρ, y) half-plane, where ρ is the distance to the axis. The profile consists of a quarter circle, a straight segment and another quarter circle, of lengths $q = \pi r/2$, $2L$ and q .

We let v be proportional to the arc length s along the profile. This choice is the reason the capsule looks so good when textured: equal steps of v are equal distances on the surface, so the texture is not squeezed near the junctions between the hemispheres and the cylinder. For each piece we compute the profile point (ρ, y) , the profile normal (n_ρ, n_y) , and the unit tangent of the profile $(d\rho/ds, dy/ds)$; the revolution by the azimuth φ then gives the point, the normal and $\partial\mathbf{P}/\partial v$.

```

⟨ Patches of the capsule 20 ⟩ ≡
std::vector<Patch> CapsulePatches() {
    constexpr float r = 0.5f, L = 0.5f;           // radius, half length of the cylinder
    constexpr float q = kPi * r / 2;             // arc length of a quarter circle
    constexpr float total = 2 * q + 2 * L;        // v is proportional to profile arc length
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float s = b * total;
        float radial, y, nr, ny, dr, dy;          // profile point, normal, and d/ds
        ⟨ Find the point of the capsule's profile at arc length s 21 ⟩
        const float ph = 2 * kPi * a, sp = std::sin(ph), cp = std::cos(ph);
        return Make({radial * cp, y, -radial * sp}, {nr * cp, ny, -nr * sp}, {-sp, 0, -cp},
            {dr * cp, dy, -dr * sp}, a, b);
    };
    return {side};
}

```

This code is used in section 14.

21. On the upper hemisphere the angle from the axis is $\theta = s/r$; on the lower one it continues from $\pi/2$. On a circle of radius r centred on the axis the normal is $(\sin\theta, \cos\theta)$ and the tangent in the direction of increasing s is $(\cos\theta, -\sin\theta)$.

```

⟨ Find the point of the capsule's profile at arc length s 21 ⟩ ≡
if (s < q) {
    const float th = s / r;
    radial = r * std::sin(th); y = L + r * std::cos(th);
    nr = std::sin(th); ny = std::cos(th); dr = std::cos(th); dy = -std::sin(th);
} else if (s < q + 2 * L) {
    radial = r; y = L - (s - q);
    nr = 1; ny = 0; dr = 0; dy = -1;
} else {
    const float th = kPi / 2 + (s - q - 2 * L) / r;
    radial = r * std::sin(th); y = -L + r * std::cos(th);
    nr = std::sin(th); ny = std::cos(th); dr = std::cos(th); dy = -std::sin(th);
}

```

This code is used in section 20.

22. The pyramid has a square base of half-side $a = 0.8$ at $y = -0.8$ and its apex at $y = 0.8$. Its four sides are flat triangles, and here the texture mapping requires a decision. We could repeat the whole texture on every face, but then a texel would appear four times on the solid, and the inspector could not say *where* it is. Instead each face receives one quarter of the texture in the u direction, while v runs from the apex ($v = 0$) to the base ($v = 1$). The price is visible: the texture is compressed horizontally and converges to a point at the apex—a good occasion to discuss why real models are “unwrapped” by artists.

The corners $c_0, \dots, c_3$ are listed so that, seen from outside, each face goes from left to right; face i spans the edge from c_i to c_{i+1} .

```

< Patches of the pyramid 22 > ≡
std::vector<Patch> PyramidPatches() {
    constexpr float a = 0.8f, h = 0.8f;
    static const XMFLOAT3 apex{0, h, 0};
    static const XMFLOAT3 c[4] = {{-a, -h, a}, {a, -h, a}, {a, -h, -a}, {-a, -h, -a}};
    std::vector<Patch> patches;
    < Add the four sides of the pyramid 23 >
    < Add the base of the pyramid 24 >
    return patches;
}

```

This code is used in section 14.

23. A side face is the family of segments from the apex $\mathbf{A}$ to the points $\mathbf{E}(s) = \text{lerp}(c_i, c_{i+1}, s)$ of the base edge: $\mathbf{P}(s, t) = \mathbf{A} + t(\mathbf{E}(s) - \mathbf{A})$. Then $\partial \mathbf{P} / \partial s = t(c_{i+1} - c_i)$; again we drop the scalar factor, which vanishes at the apex. The normal is the same for the whole face; we compute it from the edge direction and the median from the apex to the middle of the edge, in the order that makes it point outwards ($\mathbf{N} = \mathbf{B} \times \mathbf{T}$, which is our rule $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$ read backwards).

```

< Add the four sides of the pyramid 23 > ≡
// Each side face owns a quarter of the texture in u; v runs from apex (0) to base (1).
for (int i = 0; i < 4; ++i) {
    Patch f;
    f.invertible = true;
    f.u0 = i / 4.0f; f.u1 = (i + 1) / 4.0f;
    f.eval = [i](float s, float t) {
        const XMVECTOR A = XMLoadFloat3(&apex), C0 = XMLoadFloat3(&c[i]), C1 =
            ↪ XMLoadFloat3(&c[(i + 1) % 4]);
        const XMVECTOR edge = XMVectorLerp(C0, C1, s);
        const XMVECTOR P = A + t * (edge - A);
        const XMVECTOR du = C1 - C0, dv = edge - A;
        const XMVECTOR N = XMVector3Normalize(XMVector3Cross(0.5f * (C0 + C1) - A, du));
        SurfacePoint sp;
        XMStoreFloat3(&sp.pos, P); XMStoreFloat3(&sp.normal, N);
        XMStoreFloat3(&sp.dPdu, du); XMStoreFloat3(&sp.dPdv, dv);
        sp.uv = {(i + s) / 4.0f, t};
        return sp;
    };
    patches.push_back(f);
}

```

This code is used in section 22.

24. The base, seen from below, is textured like the bottom cap of the cylinder: a planar projection that is not invertible.

```

< Add the base of the pyramid 24 > ≡
Patch base;
base.eval = [](float s, float t) {
    const float x = -a + 2 * a * s, z = a - 2 * a * t;
    return Make({x, -h, z}, {0, -1, 0}, {1, 0, 0}, {0, 0, -1}, s, t);
};
patches.push_back(base);

```

This code is used in section 22.

25. Now the public functions. The first two are mere dispatchers.

```

< Public functions of the mesh module 25 > ≡
const char* ShapeName(Shape s) {
    switch (s) {
        case Shape::Sphere: return "Sphere";
        case Shape::Pyramid: return "Pyramid";
        case Shape::Cylinder: return "Cylinder";
        case Shape::Capsule: return "Capsule";
        default: return "?";
    }
}

```

See also sections 26, 27, 28 and 31. This code is used in section 14.

26.

```

<Public functions of the mesh module 25> +=
std::vector<Patch> MakePatches(Shape shape) {
    switch (shape) {
        case Shape::Sphere: return SpherePatches();
        case Shape::Pyramid: return PyramidPatches();
        case Shape::Cylinder: return CylinderPatches();
        case Shape::Capsule: return CapsulePatches();
        default: return {};
    }
}

```

27. *The tangent frame.* From a surface point we build an orthonormal frame ($\mathbf{T}, \mathbf{B}, \mathbf{N}$) by the Gram–Schmidt process: $\mathbf{N}$ is normalized, $\mathbf{T}$ is $\partial \mathbf{P} / \partial u$ minus its component along $\mathbf{N}$, normalized, and $\mathbf{B} = \pm \mathbf{N} \times \mathbf{T}$, with the sign chosen so that $\mathbf{B}$ points in the direction of increasing v . If $\partial \mathbf{P} / \partial u$ happens to be parallel to $\mathbf{N}$ or zero we fall back to a direction derived from $\partial \mathbf{P} / \partial v$. The shader performs exactly the same construction (see *<Rebuild the tangent frame in the pixel shader 73>*), so that the inspector and the GPU agree.

```

<Public functions of the mesh module 25> +=
void TangentFrame(const SurfacePoint& p, XMFLOAT3& T, XMFLOAT3& B, XMFLOAT3& N) {
    const XMVECTOR n = XMVector3Normalize(XMLoadFloat3(&p.normal));
    const XMVECTOR du = XMLoadFloat3(&p.dPdu), dv = XMLoadFloat3(&p.dPdv);
    XMVECTOR t = du - n * XMVector3Dot(n, du);
    if (XMVectorGetX(XMVector3LengthSq(t)) < 1e-12f) // degenerate: derive from dv instead
        t = XMVector3Cross(dv, n);
    t = XMVector3Normalize(t);
    XMVECTOR b = XMVector3Cross(n, t);
    if (XMVectorGetX(XMVector3Dot(b, dv)) < 0) b = -b;
    XMStoreFloat3(&T, t); XMStoreFloat3(&B, b); XMStoreFloat3(&N, n);
}

```

28. *Building a mesh.* Every patch is sampled on a regular grid of $(\text{segments}+1)^2$ points; the renderer asks for 256 segments, which gives enough vertices for displacement mapping to show detail. Patches share no vertices, which is the simplest choice and also the correct one where normals differ, as along the edges of the pyramid.

```

<Public functions of the mesh module 25> +=
MeshData BuildMesh(Shape shape, int segments) {
    MeshData mesh;
    for (const Patch& patch : MakePatches(shape)) {
        const uint32_t base = static_cast<uint32_t>(mesh.vertices.size());
        <Emit the vertices of one patch 29>
        <Emit the triangles of one patch 30>
    }
    return mesh;
}

```

29. For every grid point we evaluate the patch, build the tangent frame and compress it into the *Vertex* layout of *<Vertex layout 10>*, computing the handedness w .

```

<Emit the vertices of one patch 29> =
for (int j = 0; j <= segments; ++j) {
    for (int i = 0; i <= segments; ++i) {
        const SurfacePoint sp = patch.eval(float(i) / segments, float(j) / segments);
        XMFLOAT3 T, B, N;
        TangentFrame(sp, T, B, N);
        // Handedness: B = w * cross(N, T)
        const XMVECTOR nxT = XMVector3Cross(XMLoadFloat3(&N), XMLoadFloat3(&T));
        const float w = XMVectorGetX(XMVector3Dot(nxT, XMLoadFloat3(&B))) < 0 ? -1.0f :
            ↪ 1.0f;
        mesh.vertices.push_back({sp.pos, N, {T.x, T.y, T.z, w}, sp.uv});
    }
}

```

This code is used in section 28.

30. Each cell (i, j) of the grid, with corners i_0, i_1 (top) and i_2, i_3 (bottom), becomes two triangles. Their orientation does not matter, because the renderer draws both faces of every triangle.

```

<Emit the triangles of one patch 30> =
const uint32_t row = segments + 1;
for (int j = 0; j < segments; ++j) {
    for (int i = 0; i < segments; ++i) {

```

```

        const uint32_t i0 = base + j * row + i, i1 = i0 + 1, i2 = i0 + row, i3 = i2 + 1;
        mesh.indices.insert(mesh.indices.end(), {i0, i2, i1, i1, i2, i3});
    }
}

```

This code is used in section 28.

31. Inverting the texture mapping. To find the point with texture coordinates (u, v) , we look for an invertible patch whose rectangle $[u_0, u_1] \times [v_0, v_1]$ contains them, solve the affine equations for (a, b) , and evaluate the patch. This is all the inspector needs to place its arrows on the solid.

```

⟨Public functions of the mesh module 25⟩ +=
bool FindSurfacePoint(Shape shape, float u, float v, SurfacePoint& out) {
    for (const Patch& patch : MakePatches(shape)) {
        if (!patch.invertible) continue;
        if (u < patch.u0 || u > patch.u1 || v < patch.v0 || v > patch.v1) continue;
        out = patch.eval((u - patch.u0) / (patch.u1 - patch.u0), (v - patch.v0) / (patch.v1
            ↪ patch.v0));
        return true;
    }
    return false;
}

```

32. Materials and their maps.

A *material* in this program is a set of up to five images of the same size, all indexed by the same texture coordinates:

Color (albedo) the base colour of the surface, stored in sRGB;

Bump a height field used only to tilt the shading normal;

Displacement a height field used to move the vertices;

Specular roughness a scalar in $[0, 1]$ that controls the size of highlights;

Normal map the normal already computed by the authors of the material; we never shade with it, but the inspector decodes it for comparison.

The materials are described by a *manifest*, a small JSON file written by the download script (§50). For each material it gives a name, its origin (site, page, author and licence) and the relative paths of the available maps.

```
< File src/Material.h 32 >≡
#pragma once
#include <d3d11.h>
#include <wrl/client.h>
#include <DirectXMath.h>
#include <filesystem>
#include <memory>
#include <string>
#include <vector>
```

```
using Microsoft::WRL::ComPtr;
```

```
< Kinds of maps 33 >
```

```
< Map images 34 >
```

```
< Materials 35 >
```

```
< The material library 36 >
```

This code is written to the file `src/Material.h`.

33. The kinds of maps are used as array indices, so a plain `enum` is more convenient here than an `enum class`.

```
< Kinds of maps 33 >≡
enum MapKind { MapColor, MapBump, MapDisplacement, MapRoughness, MapNormal, MapCount };
const char* MapName(MapKind k);
```

This code is used in section 32.

34. Every image lives twice. The GPU copy is a texture with a complete chain of mipmaps, seen through two *views*: a linear one, which returns the stored bytes divided by 255, and an sRGB one, which converts colours to linear light while sampling, as physically based lighting requires. Data maps (heights, roughness) must *not* be converted, so for them both views are the same. The user interface always uses the linear view, because it wants to show the image as it is stored.

The CPU copy, an array of bytes, is what the inspector reads. Keeping it costs four megabytes per 1024×1024 map, a small price for being able to compute, without asking the GPU, exactly what the GPU computes.

```
< Map images 34 >≡
// One texture: kept both on the GPU (for rendering / UI) and on the CPU (for the
// ↪ inspector).
struct MapImage {
    int width = 0, height = 0;
    std::vector<uint8_t> rgba; // CPU copy, 8 bits per channel
    ComPtr<ID3D11Texture2D> texture;
    ComPtr<ID3D11ShaderResourceView> srvLinear; // raw values (data maps, UI display)
    ComPtr<ID3D11ShaderResourceView> srvSrgb; // color decoded from sRGB (shading)

    // Bilinear sample with wrap addressing; channels in [0,1].
    DirectX::XMFLLOAT4 Sample(float u, float v) const;
    // Nearest texel fetch with wrap addressing (red channel).
```

```
float Texel(int x, int y) const;
};
```

This code is used in section 32.

35. A material holds the information of its manifest entry and, once loaded, its maps. The maps are shared pointers because one image may play two roles: when a pack has no bump map, the displacement map is used as the bump map (§49).

```
< Materials 35 > ≡
struct Material {
    std::string id, name, site, url, author, license;
    std::string files[MapCount];
    bool bumpFromHeight = true;
    bool loaded = false;
    std::shared_ptr<MapImage> maps[MapCount];    // may be null when a map is not available
};
```

This code is used in section 32.

36. The library reads the manifest once and loads the images of a material when they are first needed.

```
< The material library 36 > ≡
class MaterialLibrary {
public:
    bool Load(const std::filesystem::path& materialsDir, std::string& error);
    // Lazily uploads a material's textures the first time it is selected.
    void EnsureLoaded(int index, ID3D11Device* device, ID3D11DeviceContext* ctx);
    std::vector<Material>& Items() { return items_; }

private:
    std::filesystem::path dir_;
    std::vector<Material> items_;
};
```

This code is used in section 32.

37. The implementation uses two small libraries: Sean Barrett's `stb_image` to decode JPEG and PNG files, and Niels Lohmann's `nlohmann/json` to read the manifest.

```
< File src/Material.cpp 37 > ≡
#include "Material.h"
#include <cmath>
#include <cstdio>
#include <fstream>
#include <nlohmann/json.hpp>

< Instantiate the image decoder 38 >

using namespace DirectX;

< Names of the maps 39 >

< Texel access with wrap-around 40 >

< Bilinear sampling on the CPU 41 >

< Load one image into a texture 42 >

< Read the manifest 47 >

< Load the maps of a material on first use 49 >
```

This code is written to the file `src/Material.cpp`.

38. `stb_image` is a single header. Exactly one source file of the program must define the macro that turns on its implementation before including it, and we only need two of its many decoders.

```
< Instantiate the image decoder 38 > ≡
#define STB_IMAGE_IMPLEMENTATION
#define STBI_ONLY_JPEG
#define STBI_ONLY_PNG
#include <stb_image.h>
```

This code is used in section 37.

39.

```

⟨Names of the maps 39⟩≡
const char* MapName(MapKind k) {
    static const char* names[] = {"Color (albedo)", "Bump", "Displacement", "Specular
    ↪roughness", "Normal map (reference)"};
    return names[k];
}

```

This code is used in section 37.

40. Textures *repeat*: the texel to the right of the last column is the first column. In C++ the remainder of a negative number is negative, hence the double % in Wrap.

```

⟨Texel access with wrap-around 40⟩≡
static int Wrap(int i, int n) { return ((i % n) + n) % n; }

float MapImage::Texel(int x, int y) const {
    return rgba[(size_t(Wrap(y, height)) * width + Wrap(x, width)) * 4] / 255.0f;
}

```

This code is used in section 37.

41. *Bilinear sampling* must agree with the GPU, so we must know the GPU's convention: the centre of texel i is at $u = (i + \frac{1}{2})/W$. A coordinate u therefore lies between the texels $x_0 = \lfloor uW - \frac{1}{2} \rfloor$ and $x_0 + 1$, with weight $f_x = uW - \frac{1}{2} - x_0$ for the second one; the same holds vertically. The result interpolates first along x on two rows and then along y .

```

⟨Bilinear sampling on the CPU 41⟩≡
XMFLOAT4 MapImage::Sample(float u, float v) const {
    // Same convention as the GPU: texel centers sit at (i + 0.5) / size.
    const float x = u * width - 0.5f, y = v * height - 0.5f;
    const int x0 = int(std::floor(x)), y0 = int(std::floor(y));
    const float fx = x - x0, fy = y - y0;
    float out[4] = {};
    for (int c = 0; c < 4; ++c) {
        auto at = [&](int xi, int yi) {
            return rgba[(size_t(Wrap(yi, height)) * width + Wrap(xi, width)) * 4 + c] /
            ↪255.0f;
        };
        const float top = at(x0, y0) * (1 - fx) + at(x0 + 1, y0) * fx;
        const float bot = at(x0, y0 + 1) * (1 - fx) + at(x0 + 1, y0 + 1) * fx;
        out[c] = top * (1 - fy) + bot * fy;
    }
    return {out[0], out[1], out[2], out[3]};
}

```

This code is used in section 37.

42. *Loading an image* has two halves: decoding the file into memory, and creating the GPU texture from that memory.

```

⟨Load one image into a texture 42⟩≡
static std::shared_ptr<MapImage> LoadImage(const std::filesystem::path& path, bool srgb,
                                             ID3D11Device* device, ID3D11DeviceContext* ctx) {

    ⟨Decode the image file 43⟩

    ⟨Keep a CPU copy of the pixels 44⟩

    ⟨Create a texture with a full mipmap chain 45⟩

    ⟨Create the linear and sRGB views 46⟩
    ctx->UpdateSubresource(img->texture.Get(), 0, nullptr, img->rgba.data(), w * 4, 0);
    ctx->GenerateMips(img->srvLinear.Get());
    return img;
}

```

This code is used in section 37.

43. The file is opened with a wide-character name, so that paths with accents work on Windows. We always ask for four channels: a grey JPEG is then expanded to grey RGB, which keeps all the code that follows uniform.

```

⟨Decode the image file 43⟩≡
FILE* f = nullptr;

```

```

_wfopen_s(&f, path.c_str(), L"rb");
if (!f) return nullptr;
int w = 0, h = 0, n = 0;
stbi_uc* pixels = stbi_load_from_file(f, &w, &h, &n, 4);
fclose(f);
if (!pixels) return nullptr;

```

This code is used in section 42.

44.

```

⟨ Keep a CPU copy of the pixels 44 ⟩ ≡
auto img = std::make_shared<MapImage>();
img->width = w;
img->height = h;
img->rgba.assign(pixels, pixels + size_t(w) * h * 4);
stbi_image_free(pixels);

```

This code is used in section 42.

45. The texture is created with a *typeless* format. This is what allows two views with different interpretations (UNORM and UNORM_SRGB) of the same bytes. Asking for `MipLevels = 0` means “the whole chain”, and the flags `RENDER_TARGET` and `GENERATE_MIPS` let the GPU compute the smaller levels itself.

```

⟨ Create a texture with a full mipmap chain 45 ⟩ ≡
// Typeless storage lets the same texture be viewed as linear (UI) or sRGB (shading).
D3D11_TEXTURE2D_DESC td{};
td.Width = w;
td.Height = h;
td.MipLevels = 0; // full chain
td.ArraySize = 1;
td.Format = DXGI_FORMAT_R8G8B8A8_TYPELESS;
td.SampleDesc.Count = 1;
td.Usage = D3D11_USAGE_DEFAULT;
td.BindFlags = D3D11_BIND_SHADER_RESOURCE | D3D11_BIND_RENDER_TARGET;
td.MiscFlags = D3D11_RESOURCE_MISC_GENERATE_MIPS;
if (FAILED(device->CreateTexture2D(&td, nullptr, &img->texture))) return nullptr;

```

This code is used in section 42.

46.

```

⟨ Create the linear and sRGB views 46 ⟩ ≡
D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
sd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
sd.Texture2D.MipLevels = UINT(-1);
sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
device->CreateShaderResourceView(img->texture.Get(), &sd, &img->srvLinear);
if (srgb) {
    sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
    device->CreateShaderResourceView(img->texture.Get(), &sd, &img->srvSrgb);
} else {
    img->srvSrgb = img->srvLinear;
}

```

This code is used in section 42.

47. *Reading the manifest.* The manifest is an array of objects; missing fields get harmless defaults. Any error is reported as text, which the user interface shows instead of the viewport—typically because the download script has not been run yet.

```

⟨ Read the manifest 47 ⟩ ≡
bool MaterialLibrary::Load(const std::filesystem::path& materialsDir, std::string& error) {
    dir_ = materialsDir;
    std::ifstream in(materialsDir / "manifest.json");
    if (!in) {
        error = "Cannot open " + (materialsDir / "manifest.json").string() +
            "\nRun tools\\fetch_textures.ps1 first.";
        return false;
    }
    try {
        ⟨ Parse the entries of the manifest 48 ⟩
    } catch (const std::exception& ex) {
        error = std::string("manifest.json: ") + ex.what();
    }
}

```

```

        return false;
    }
    if (items_.empty()) error = "manifest.json lists no materials";
    return !items_.empty();
}

```

This code is used in section 37.

48.

```

< Parse the entries of the manifest 48 >≡
const nlohmann::json j = nlohmann::json::parse(in);
static const char* keys[MapCount] = {"color", "bump", "displacement", "roughness",
    ↪ "normal"};
for (const auto& e : j) {
    Material m;
    m.id = e.value("id", "");
    m.name = e.value("name", m.id);
    m.site = e.value("site", "");
    m.url = e.value("url", "");
    m.author = e.value("author", "");
    m.license = e.value("license", "");
    m.bumpFromHeight = e.value("bumpFromHeight", true);
    if (e.contains("files"))
        for (int k = 0; k < MapCount; ++k)
            m.files[k] = e["files"].value(keys[k], "");
    items_.push_back(std::move(m));
}

```

This code is used in section 47.

49. *Loading the maps of a material.* The paths in the manifest are UTF-8; C++20 wants them as `char8_t` to interpret them so. Only the colour map is given an sRGB view.

Then comes a decision that matters for teaching. Neither ambientCG nor Poly Haven ships a separate bump map for these materials, and this is no accident: a bump map *is* a height field, and the displacement map is the height field of the material. So when no bump map is present we use the displacement map in its place, and remember that we did, so that the interface can say so. The same data then plays both roles, and switching bump and displacement on and off separately shows clearly what each one does.

```

< Load the maps of a material on first use 49 >≡
void MaterialLibrary::EnsureLoaded(int index, ID3D11Device* device, ID3D11DeviceContext*
    ↪ ctx) {
    Material& m = items_[index];
    if (m.loaded) return;
    m.loaded = true;
    for (int k = 0; k < MapCount; ++k)
        if (!m.files[k].empty())
            m.maps[k] = LoadImage(dir_ / std::filesystem::path(reinterpret_cast<const
                ↪ char8_t*>(m.files[k].c_str())), k == MapColor, device, ctx);
    // A bump map is a height field used only for shading. When a pack has no separate
    // one, the height (displacement) map is the bump map.
    if (!m.maps[MapBump]) {
        m.maps[MapBump] = m.maps[MapDisplacement];
        m.bumpFromHeight = true;
    } else {
        m.bumpFromHeight = false;
    }
}

```

This code is used in section 37.

50. *Collecting the materials.* The ten example materials are not part of the program; a PowerShell script downloads them and writes the manifest. Seven come from ambientCG, whose web API returns, for a list of asset identifiers, the address of a ZIP file for each resolution; three come from Poly Haven, whose API returns the address of each map separately. All of them are in the public domain (CC0).

```

< File tools/fetch_textures.ps1 50 >≡
< Description and parameters of the download script 51 >

< Choose the ten materials 52 >

< Download the ambientCG materials 53 >

```

⟨ *Download the Poly Haven materials* 54 ⟩

⟨ *Write the manifest* 55 ⟩

This code is written to the file `tools/fetch_textures.ps1`.

51. The script can be run again safely: files that already exist are not downloaded again, unless `-Force` is given.

⟨ *Description and parameters of the download script* 51 ⟩ ≡

```
<#
Downloads a small, curated set (10) of CCO PBR materials from ambientCG and
Poly Haven, normalizes their file names and writes assets/materials/manifest.json.

Every material keeps: color, displacement, roughness and (reference) normal map.
Neither site ships a separate "bump" map for these assets; the viewer therefore
derives the bump map from the height/displacement map and labels it as such.

Usage: powershell -ExecutionPolicy Bypass -File tools\fetch_textures.ps1 [-Force]
#>
param([switch]$Force)

$ErrorActionPreference = 'Stop'
$ProgressPreference = 'SilentlyContinue'
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$root = Join-Path (Split-Path -Parent $PSScriptRoot) 'assets\materials'
New-Item -ItemType Directory -Force $root | Out-Null
This code is used in section 50.
```

52. The selection tries to cover different kinds of surfaces: masonry with deep mortar lines, wood, rock, metal, glazed tiles, woven fabric, paving, and bark. Each kind exercises bump and roughness maps differently: the glazed tiles, for example, alternate mirror-like and rough regions.

⟨ *Choose the ten materials* 52 ⟩ ≡

```
$ambientCG = @('Bricks076C', 'Wood066', 'Rock030', 'Metal032', 'Tiles101', 'Fabric048',
    ↳ 'PavingStones131')
$polyHaven = @('castle_brick_07', 'aerial_rocks_02', 'bark_willow_02')

$manifest = @()

function Save-Url($url, $path) {
    if ($Force -or -not (Test-Path $path)) {
        Write-Host " GET $url"
        Invoke-WebRequest -UseBasicParsing -Uri $url -OutFile $path
    }
}
```

This code is used in section 50.

53. For ambientCG we ask the API once for all seven assets, then for each one download the 1K-JPG archive, extract it to a temporary folder, and copy the four maps we use under standard names. We take the DirectX variant of the normal map, whose green channel points to $+v$ like our tangent-space y axis.

⟨ *Download the ambientCG materials* 53 ⟩ ≡

```
# ----- ambientCG
$sids = $ambientCG -join ','
$api = Invoke-WebRequest -UseBasicParsing
    ↳ "https://ambientcg.com/api/v2/full_json?id=$sids&include=downloadData" |
    Select-Object -ExpandProperty Content | ConvertFrom-Json
foreach ($id in $ambientCG) {
    $asset = $api.foundAssets | Where-Object assetId -eq $id
    if (-not $asset) { Write-Warning "ambientCG asset $id not found"; continue }
    Write-Host "ambientCG: $id"
    $dir = Join-Path $root $id
    New-Item -ItemType Directory -Force $dir | Out-Null
    $dl = $asset.downloadFolders.default.downloadFiletypeCategories.zip.downloads |
        ↳ Where-Object attribute -eq '1K-JPG'
    $zip = Join-Path $env:TEMP "$id_1K-JPG.zip"
    $tmp = Join-Path $env:TEMP "texex_$id"
```

```

if ($Force -or -not (Test-Path (Join-Path $dir 'color.jpg'))) {
    Save-Url $dl.fullDownloadPath $zip
    if (Test-Path $tmp) { Remove-Item -Recurse -Force $tmp }
    Expand-Archive -Path $zip -DestinationPath $tmp
    $map = @{ 'Color' = 'color.jpg'; 'Displacement' = 'displacement.jpg'; 'Roughness' =
        ↳ 'roughness.jpg'; 'NormalDX' = 'normal.jpg' }
    foreach ($k in $map.Keys) {
        $src = Get-ChildItem $tmp -Filter "*_$k.jpg" | Select-Object -First 1
        if ($src) { Copy-Item $src.FullName (Join-Path $dir $map[$k]) -Force }
    }
    Remove-Item -Recurse -Force $tmp
    Remove-Item -Force $zip
}
$manifest += [ordered]@{
    id      = $id
    name    = $asset.displayName
    site    = 'ambientCG'
    url     = "https://ambientcg.com/view?id=$id"
    author  = 'Lennart Demes (ambientCG)'
    license = 'CC0 1.0'
}
}

```

This code is used in section 50.

54. Poly Haven needs two requests per asset: one for the list of files, one for the descriptive information, which includes the authors.

```

< Download the Poly Haven materials 54 > ≡
# ----- Poly Haven
foreach ($id in $polyHaven) {
    Write-Host "Poly Haven: $id"
    $files = Invoke-WebRequest -UseBasicParsing "https://api.polyhaven.com/files/$id" |
        ↳ Select-Object -ExpandProperty Content | ConvertFrom-Json
    $info = Invoke-WebRequest -UseBasicParsing "https://api.polyhaven.com/info/$id" |
        ↳ Select-Object -ExpandProperty Content | ConvertFrom-Json
    $dir = Join-Path $root $id
    New-Item -ItemType Directory -Force $dir | Out-Null
    $map = @{ 'Diffuse' = 'color.jpg'; 'Displacement' = 'displacement.jpg'; 'Rough' =
        ↳ 'roughness.jpg'; 'nor_dx' = 'normal.jpg' }
    foreach ($k in $map.Keys) {
        $url = $files.$k.'1k'.jpg.url
        if ($url) { Save-Url $url (Join-Path $dir $map[$k]) }
    }
    $manifest += [ordered]@{
        id      = $id
        name    = $info.name
        site    = 'Poly Haven'
        url     = "https://polyhaven.com/a/$id"
        author  = ($info.authors.PSObject.Properties.Name -join ', ')
        license = 'CC0 1.0'
    }
}

```

This code is used in section 50.

55. Finally the list of files that actually exist is recorded for every material, together with the flag that tells the program to use the height map as bump map.

```

< Write the manifest 55 > ≡
# ----- manifest
foreach ($m in $manifest) {
    $dir = Join-Path $root $m.id
    $m.files = [ordered]@{}
    foreach ($f in 'color', 'bump', 'displacement', 'roughness', 'normal') {
        $p = Join-Path $dir "$f.jpg"
        if (Test-Path $p) { $m.files[$f] = "$($m.id)/$f.jpg" }
    }
    # No pack provides a dedicated bump map: bump is derived from the height map.
    $m.bumpFromHeight = -not $m.files.Contains('bump')
}

```

```
}  
$json = ConvertTo-Json -InputObject @($manifest) -Depth 5  
[IO.File]::WriteAllText((Join-Path $root 'manifest.json'), $json, (New-Object  
    ⇨Text.UTF8Encoding $false))  
Write-Host "Wrote $($manifest.Count) materials to $root\manifest.json"  
This code is used in section 50.
```

56. The probe: from a texel to a normal.

This part is the mathematical heart of the program. Given a material, a texel (u, v) and the current shape and orientation of the solid, the *inspector* computes

- the values of all maps at (u, v) ;
- the slope of the bump map there, and the normal $\mathbf{n}_{ts}$ that this slope implies, in tangent space;
- the point of the solid that carries this texel, the frame $(\mathbf{T}, \mathbf{B}, \mathbf{N})$ there, and the bumped normal $\mathbf{n}'$ in world space.

The GPU does the same computation for every pixel it draws; the inspector does it once, on the CPU, and shows every intermediate result.

```
< File src/Inspector.h 56 >≡
#pragma once
#include "Material.h"
#include "Mesh.h"
```

```
< Settings of bump and displacement 57 >
```

```
< What the probe reports 58 >
```

```
< Interface of the inspector 59 >
```

This code is written to the file `src/Inspector.h`.

57. The parameters that the user controls with sliders, and that both the inspector and the shader must know, are gathered in one structure. The *mid-level* of the displacement is the height that leaves the surface where it is: with the default 0.5, darker texels push the surface in and lighter ones push it out.

```
< Settings of bump and displacement 57 >≡
struct BumpSettings {
    float strength = 4.0f;           // slope multiplier, per texel of the bump map
    float displacementScale = 0.05f;
    float displacementMid = 0.5f;   // height value that leaves the surface in place
    bool useDisplacement = true;
};
```

This code is used in section 56.

58. The answer of the probe. The world-space part is only meaningful when the texel lies on an invertible patch of the current shape (§31).

```
< What the probe reports 58 >≡
struct HoverInfo {
    bool valid = false;
    DirectX::XMFLOAT2 uv{};
    DirectX::XMFLOAT3 color{};
    float bump = 0, displacement = 0, roughness = 0;
    DirectX::XMFLOAT2 slope{};      // dh/dx, dh/dy in texels (central differences)
    DirectX::XMFLOAT3 normalTangent{};
    // World-space quantities, available when the uv lands on the current shape.
    bool onSurface = false;
    DirectX::XMFLOAT3 position{}, normalGeom{}, normalBumped{}, tangent{}, bitangent{};
};
```

This code is used in section 56.

59.

```
< Interface of the inspector 59 >≡
// Normal from a height (bump) map, in tangent space. This is the exact formula the
// pixel shader uses (see shaders/pbr.hlsl, BumpNormalTS).
DirectX::XMFLOAT3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength,
                                   DirectX::XMFLOAT2* slopeOut = nullptr);

HoverInfo Inspect(const Material& m, float u, float v, Shape shape, DirectX::FXMMATRIX
    ↪ world,
                const BumpSettings& s);
```

This code is used in section 56.

60. The implementation consists of the two functions just declared.

```
< File src/Inspector.cpp 60 > ≡
#include "Inspector.h"
```

```
using namespace DirectX;
```

```
< The normal of a height field 61 >
```

```
< Inspect one texel 62 >
```

This code is written to the file `src/Inspector.cpp`.

61. *The normal of a height field.* Imagine that, in the neighbourhood of a point, the surface is the flat plane of tangent space, measured in texels, and that the bump map lifts it to the height $z = k h(x, y)$, where k is the *bump strength*. The lifted surface has the two tangent vectors

$$\mathbf{t}_x = (1, 0, k h_x), \quad \mathbf{t}_y = (0, 1, k h_y),$$

where h_x and h_y are the partial derivatives of h . The vector

$$\mathbf{n}_{ts} \propto (-k h_x, -k h_y, 1)$$

is perpendicular to both, as one checks at once: $\mathbf{n}_{ts} \cdot \mathbf{t}_x = -k h_x + k h_x = 0$, and likewise for $\mathbf{t}_y$. (We avoid saying “the cross product of $\mathbf{t}_x$ and $\mathbf{t}_y$ ”, because our tangent frame is left-handed—recall $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$ —and the orthogonality argument does not care about handedness.) Where the map is flat, $h_x = h_y = 0$ and $\mathbf{n}_{ts} = (0, 0, 1)$: the unperturbed normal.

The derivatives are estimated by *central differences* one texel to each side,

$$h_x \approx \frac{1}{2}(h(x+1, y) - h(x-1, y)), \quad h_y \approx \frac{1}{2}(h(x, y+1) - h(x, y-1)),$$

which is exact for quadratic functions and, unlike a one-sided difference, does not shift the normal by half a texel. The samples are bilinear, so the result varies smoothly between texels. The pixel shader evaluates exactly the same expression (see *< The normal of a height field, on the GPU 70 >*).

```
< The normal of a height field 61 > ≡
```

```
XMFLOAT3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength, XMFLOAT2*
    ↪slopeOut) {
    const float du = 1.0f / bump.width, dv = 1.0f / bump.height;
    // Height change per texel, by central differences.
    const float dhdx = 0.5f * (bump.Sample(u + du, v).x - bump.Sample(u - du, v).x);
    const float dhdy = 0.5f * (bump.Sample(u, v + dv).x - bump.Sample(u, v - dv).x);
    if (slopeOut) *slopeOut = {dhdx, dhdy};
    // The surface z = k*h(x,y) has normal (-k*dh/dx, -k*dh/dy, 1).
    XMFLOAT3 n;
    XMStoreFloat3(&n, XMVector3Normalize(XMVectorSet(-strength * dhdx, -strength * dhdy, 1,
        ↪0)));
    return n;
}
```

This code is used in section 60.

62. *Inspecting a texel.* First the values that depend only on the material.

```
< Inspect one texel 62 > ≡
```

```
HoverInfo Inspect(const Material& m, float u, float v, Shape shape, FXMMATRIX world, const
    ↪BumpSettings& s) {
    HoverInfo h;
    h.valid = true;
    h.uv = {u, v};
    < Sample every map at the probe 63 >

    < Find the probe on the solid and express the normals in world space 64 >
    return h;
}
```

This code is used in section 60.

63. A missing map leaves its value at zero; a missing bump map leaves the normal unperturbed.

```
< Sample every map at the probe 63 > ≡
```

```
if (auto& c = m.maps[MapColor]) { const XMFLOAT4 x = c->Sample(u, v); h.color = {x.x, x.y,
    ↪x.z}; }
if (auto& b = m.maps[MapBump]) h.bump = b->Sample(u, v).x;
```

```

if (auto& d = m.maps[MapDisplacement]) h.displacement = d->Sample(u, v).x;
if (auto& r = m.maps[MapRoughness]) h.roughness = r->Sample(u, v).x;
h.normalTangent = m.maps[MapBump] ? BumpNormalTangent(*m.maps[MapBump], u, v, s.strength,
    ↪&h.slope)
                                : XMFLOAT3{0, 0, 1};

```

This code is used in section 62.

64. *From tangent space to the world.* Let $\mathbf{T}$, $\mathbf{B}$, $\mathbf{N}$ be the orthonormal frame at the point, as columns of a matrix. A vector with tangent-space coordinates (x, y, z) is, in object space,

$$\mathbf{n}_{obj} = x\mathbf{T} + y\mathbf{B} + z\mathbf{N} = (\mathbf{T} \ \mathbf{B} \ \mathbf{N}) \begin{pmatrix} x \\ y \\ z \end{pmatrix}.$$

This is the famous TBN matrix. Because it is orthonormal, its inverse is its transpose, and the reverse transformation is just three dot products—but we do not need it here.

The object is then rotated into the world by the world matrix. In general normals must be transformed by the inverse transpose of the matrix that transforms points; our world matrix is a pure rotation, for which the two coincide, so `XMVector3TransformNormal` with the world matrix itself is correct. The position also receives the displacement, so that the arrows start on the displaced surface that the user actually sees.

```

⟨ Find the probe on the solid and express the normals in world space 64 ⟩ ≡
SurfacePoint sp;
if (!FindSurfacePoint(shape, u, v, sp)) return h;
h.onSurface = true;
XMFLOAT3 T, B, N;
TangentFrame(sp, T, B, N);
const XMVECTOR t = XMLoadFloat3(&T), b = XMLoadFloat3(&B), n = XMLoadFloat3(&N);
// Tangent space -> object space: columns of the TBN matrix are T, B, N.
const XMFLOAT3& nt = h.normalTangent;
const XMVECTOR nObj = t * nt.x + b * nt.y + n * nt.z;
XMVECTOR p = XMLoadFloat3(&sp.pos);
if (s.useDisplacement && m.maps[MapDisplacement])
    p += n * (s.displacementScale * (h.displacement - s.displacementMid));
// Object -> world. The world matrix is a pure rotation, so it also transforms normals.
XMStoreFloat3(&h.position, XMVector3TransformCoord(p, world));
XMStoreFloat3(&h.normalGeom, XMVector3Normalize(XMVector3TransformNormal(n, world)));
XMStoreFloat3(&h.normalBumped, XMVector3Normalize(XMVector3TransformNormal(nObj, world)));
XMStoreFloat3(&h.tangent, XMVector3Normalize(XMVector3TransformNormal(t, world)));
XMStoreFloat3(&h.bitangent, XMVector3Normalize(XMVector3TransformNormal(b, world)));

```

This code is used in section 62.

65. The shaders.

Everything we have computed on the CPU for one texel, the GPU computes for every pixel of the image, sixty times a second. It does so in two small programs written in HLSL, the shading language of Direct3D: a *vertex shader*, run once for each vertex of the mesh, which may move the vertex (displacement mapping) and transforms it to the screen; and a *pixel shader*, run once for each covered pixel, which builds the bumped normal and evaluates the lighting.

The program compiles the shaders when it starts and whenever the user presses F5, so students can edit this file and see the result at once.

```
< File shaders/pbr.hlsl 65 > ≡
// Texture Explorer - main surface shader.
// Edit and press F5 (or "Reload shaders") to recompile while the program runs.

< Constants of a frame 66 >

< Textures and sampler 67 >

< Vertex formats of the surface shader 68 >

< The vertex shader: displacement mapping 69 >

< The normal of a height field, on the GPU 70 >

static const float PI = 3.14159265;

< Undo the display encoding of debug colours 71 >

< The pixel shader 72 >
```

This code is written to the file `shaders/pbr.hlsl`.

66. All the parameters of a frame travel in one constant buffer. HLSL packs constant buffers in rows of four floats, and a vector may not straddle two rows; the declarations are arranged accordingly, and the C++ structure that fills the buffer (*< Constants of a frame, as seen from C++ 84 >*) mirrors them field by field. DirectXMath stores matrices by rows and multiplies row vectors from the left, $\mathbf{p}' = \mathbf{p}M$; declaring the matrices `row_major` and writing `mul(p, M)` in HLSL follows the same convention and spares a transposition.

```
< Constants of a frame 66 > ≡
cbuffer Frame : register(b0)
{
    row_major float4x4 world;      // object -> world (a rotation)
    row_major float4x4 viewProj;  // world -> clip
    float3 cameraPos;   float bumpStrength;
    float3 lightPos;    float lightIntensity;
    float3 lightColor;  float dispScale;
    float dispMid;      float dispLod; float metallic; float ambient;
    int   viewMode;     int useColor;  int useBump;   int useDisp;
    int   useRough;     float constRoughness; float2 hoverUV;
    float hoverOn;     float3 pad;
};
```

This code is used in section 65.

67. The four maps used for rendering occupy the first four texture slots. A single sampler, with anisotropic filtering and wrap-around addressing, serves them all.

```
< Textures and sampler 67 > ≡
Texture2D   colorMap : register(t0); // sRGB view -> linear albedo
Texture2D   bumpMap  : register(t1); // height used only for shading
Texture2D   dispMap  : register(t2); // height used to move vertices
Texture2D   roughMap : register(t3); // specular roughness
SamplerState linearWrap : register(s0);
```

This code is used in section 65.

68. The vertex shader reads the `Vertex` structure of *< Vertex layout 10 >* and passes to the pixel shader the world position (for the lighting vectors), the normal, the tangent with its sign, and the texture coordinates. The rasterizer interpolates all of them across each triangle.

```
< Vertex formats of the surface shader 68 > ≡
struct VSIn { float3 pos : POSITION; float3 nrm : NORMAL; float4 tan : TANGENT; float2 uv :
```

```

    ↪TEXCOORD; };
    struct VSOut { float4 svpos : SV_Position; float3 wpos : WORLDPOS; float3 nrm : NORMAL;
    ↪float4 tan : TANGENT; float2 uv : TEXCOORD; };

```

This code is used in section 65.

69. *Displacement mapping* is a single line: the vertex moves along its normal by $s(h - m)$, where h is the height, s the scale and m the mid-level. A vertex shader cannot use automatic mipmap selection—there are no neighbouring pixels to compare—so it must choose a level explicitly. The C++ side computes `dispLod` so that the sampled level has about as many texels as the mesh has vertices, $\log_2(1024/256) = 2$ for our maps; sampling the full-resolution level would make the surface shimmer.

Note what this shader does *not* do: it does not change the normal. A displaced surface shaded with the normals of the undisplaced one has the right silhouette but the wrong lighting; that is why displacement maps are almost always paired with a bump or normal map, and why the program lets you switch them independently.

```

< The vertex shader: displacement mapping 69 > ≡
VSOut VSMain(VSIn i)
{
    float3 p = i.pos;
    if (useDisp != 0)
    {
        // Displacement mapping: move the vertex along its normal by the height.
        float h = dispMap.SampleLevel(linearWrap, i.uv, dispLod).r;
        p += i.nrm * dispScale * (h - dispMid);
    }
    VSOut o;
    float4 wp = mul(float4(p, 1), world);
    o.svpos = mul(wp, viewProj);
    o.wpos = wp.xyz;
    o.nrm = mul(float4(i.nrm, 0), world).xyz;
    o.tan = float4(mul(float4(i.tan.xyz, 0), world).xyz, i.tan.w);
    o.uv = i.uv;
    return o;
}

```

This code is used in section 65.

70. This is the formula of §61, word for word. The size of a texel is obtained from the texture itself, and the samples are taken from level 0, as the CPU does, so that the two computations agree exactly.

```

< The normal of a height field, on the GPU 70 > ≡
// Normal of the height field  $z = k \cdot h(x, y)$ , in tangent space, by central differences
// (one texel each side). Same formula as BumpNormalTangent() in src/Inspector.cpp.
float3 BumpNormalTS(float2 uv)
{
    float w, h;
    bumpMap.GetDimensions(w, h);
    float2 t = 1.0 / float2(w, h);
    float dhdx = 0.5 * (bumpMap.SampleLevel(linearWrap, uv + float2(t.x, 0), 0).r -
        bumpMap.SampleLevel(linearWrap, uv - float2(t.x, 0), 0).r);
    float dhdy = 0.5 * (bumpMap.SampleLevel(linearWrap, uv + float2(0, t.y), 0).r -
        bumpMap.SampleLevel(linearWrap, uv - float2(0, t.y), 0).r);
    return normalize(float3(-bumpStrength * dhdx, -bumpStrength * dhdy, 1));
}

```

This code is used in section 65.

71. The scene is rendered into an sRGB target: the GPU encodes the linear colours we compute with (approximately) the power $1/2.2$ when writing them. For the debug views we want the opposite—the value 0.5 of a normal component must appear as the byte 128—so we pre-apply the inverse power. (The sRGB curve is not exactly a power law, but the difference is invisible here.)

```

< Undo the display encoding of debug colours 71 > ≡
float3 SrgbToLinear(float3 c) { return pow(saturate(c), 2.2); } // so debug colors display
    ↪unchanged

```

This code is used in section 65.

72. The pixel shader proceeds in the order of the introduction: tangent frame, bumped normal, material values, and then either a debug view or the lighting.

```

< The pixel shader 72 > ≡
float4 PSMain(VSOut i) : SV_Target
{
    < Rebuild the tangent frame in the pixel shader 73 >

    < Bump the normal 74 >

    < Read the colour and the roughness 75 >

    < Choose a debug view 76 >

    float3 color;
    if (dbg.x >= 0)
    {
        color = SrgbToLinear(dbg);
    }
    else
    {
        < Evaluate the Cook-Torrance BRDF 77 >
    }

    < Mark the probed texel 81 >
    return float4(color, 1);
}

```

This code is used in section 65.

73. Interpolation across a triangle does not preserve lengths or right angles, so the frame is rebuilt: the normal is normalized, the tangent is made perpendicular to it again (Gram–Schmidt), and the bitangent is computed from both and the stored sign, exactly as *< Public functions of the mesh module 25 >* does on the CPU.

```

< Rebuild the tangent frame in the pixel shader 73 > ≡
// Tangent frame (TBN). B is rebuilt from N and T with the stored handedness.
float3 N = normalize(i.nrm);
float3 T = normalize(i.tan.xyz - N * dot(N, i.tan.xyz));
float3 B = i.tan.w * cross(N, T);

```

This code is used in section 72.

74. The bumped normal is the tangent-space normal multiplied by the TBN matrix, as in *< Find the probe on the solid and express the normals in world space 64 >*.

```

< Bump the normal 74 > ≡
float3 nTS = useBump != 0 ? BumpNormalTS(i.uv) : float3(0, 0, 1);
float3 n = normalize(T * nTS.x + B * nTS.y + N * nTS.z);

```

This code is used in section 72.

75. When the user switches off a map, a neutral value replaces it: a light grey albedo, or the roughness chosen with a slider. Roughness is clamped away from zero because a perfectly smooth GGX surface has an infinitely thin highlight, which a point light cannot show.

```

< Read the colour and the roughness 75 > ≡
float3 albedo = useColor != 0 ? colorMap.Sample(linearWrap, i.uv).rgb : float3(0.7, 0.7,
    ↪0.7);
float rough = useRough != 0 ? roughMap.Sample(linearWrap, i.uv).r : constRoughness;
rough = clamp(rough, 0.04, 1.0);

```

This code is used in section 72.

76. *Debug views* replace the lighting by one of the intermediate quantities, encoded as a colour. A negative value means “no debug view”. The UV view is particularly instructive on the sphere and on the pyramid.

```

< Choose a debug view 76 > ≡
float3 dbg = float3(-1, -1, -1);
if (viewMode == 1) dbg = pow(saturate(albedo), 1 / 2.2);
else if (viewMode == 2) dbg = n * 0.5 + 0.5;
else if (viewMode == 3) dbg = nTS * 0.5 + 0.5;
else if (viewMode == 4) dbg = rough.xxx;
else if (viewMode == 5) dbg = bumpMap.Sample(linearWrap, i.uv).rrr;
else if (viewMode == 6) dbg = float3(frac(i.uv), 0);

```

This code is used in section 72.

77. Lighting. We use the microfacet model of Cook and Torrance in the form that real-time graphics has made standard. The surface is imagined as a multitude of tiny mirrors (microfacets); for light to be reflected from the direction $\mathbf{L}$ to the direction $\mathbf{V}$, a facet must have its normal along the half vector $\mathbf{H} = (\mathbf{L} + \mathbf{V})/\|\mathbf{L} + \mathbf{V}\|$. The specular part of the BRDF is

$$f_s = \frac{D(\mathbf{H}) G(\mathbf{L}, \mathbf{V}) F(\mathbf{V}, \mathbf{H})}{4 (\mathbf{n} \cdot \mathbf{L})(\mathbf{n} \cdot \mathbf{V})},$$

where D is the fraction of facets oriented along $\mathbf{H}$, G the fraction of them that are neither shadowed nor masked by others, and F the Fresnel reflectance. All dot products use the *bumped* normal $\mathbf{n}$; that is where the bump map acts on the image.

```
< Evaluate the Cook-Torrance BRDF 77 > ≡
// Cook-Torrance microfacet BRDF: GGX distribution, Smith-Schlick geometry, Schlick Fresnel.
< Directions for the lighting 78 >
```

```
< The three factors of the specular term 79 >
```

```
< Add the diffuse term, the light and the ambient term 80 >
```

This code is used in section 72.

78. The camera and the light are points, so the directions vary across the surface.

```
< Directions for the lighting 78 > ≡
float3 V = normalize(cameraPos - i.wpos);
float3 L = normalize(lightPos - i.wpos);
float3 H = normalize(V + L);
float NdotL = saturate(dot(n, L));
float NdotV = max(dot(n, V), 1e-4);
float NdotH = saturate(dot(n, H));
float VdotH = saturate(dot(V, H));
```

This code is used in section 77.

79. Roughness enters here. The GGX (Trowbridge-Reitz) distribution with parameter $\alpha = r^2$, where r is the value of the roughness map, is

$$D = \frac{\alpha^2}{\pi ((\mathbf{n} \cdot \mathbf{H})^2 (\alpha^2 - 1) + 1)^2}.$$

For small α it is a tall, narrow peak—a sharp highlight—and for $\alpha = 1$ it is constant—no highlight at all, just a sheen. Squaring the map value makes the slider and the map perceptually more uniform. The geometry term is Schlick's approximation of Smith's function with $k = (r + 1)^2/8$, and the Fresnel term is Schlick's $F = F_0 + (1 - F_0)(1 - \mathbf{V} \cdot \mathbf{H})^5$, with $F_0 = 0.04$ for dielectrics and the albedo itself for metals.

```
< The three factors of the specular term 79 > ≡
float a = rough * rough;
float a2 = a * a;
float d = NdotH * NdotH * (a2 - 1) + 1;
float D = a2 / (PI * d * d);
float k = (rough + 1) * (rough + 1) / 8;
float G = (NdotV / (NdotV * (1 - k) + k)) * (NdotL / (NdotL * (1 - k) + k));
float3 F0 = lerp(float3(0.04, 0.04, 0.04), albedo, metallic);
float3 F = F0 + (1 - F0) * pow(1 - VdotH, 5);
```

This code is used in section 77.

80. The light that is not reflected specularly, $1 - F$, enters the material and comes out diffusely (Lambert, ρ/π); metals absorb it. The total is multiplied by the radiance of the light and by the cosine $\mathbf{n} \cdot \mathbf{L}$.

One more factor, *horizon*, deserves a word. A bumped normal can face the light even where the real surface faces away from it, which makes the dark side of the solid sparkle. Multiplying by a steep ramp of the *geometric* cosine $\mathbf{N} \cdot \mathbf{L}$ removes the artefact; it is a crude form of self-shadowing.

```
< Add the diffuse term, the light and the ambient term 80 > ≡
float3 spec = D * G * F / (4 * NdotV * max(NdotL, 1e-4));
float3 kd = (1 - F) * (1 - metallic);
float3 radiance = lightColor * lightIntensity;
// Geometric self-shadowing: no light where the unperturbed surface faces away.
float horizon = saturate(dot(N, L) * 8);
color = (kd * albedo / PI + spec) * radiance * NdotL * horizon + ambient * albedo;
```

This code is used in section 77.

81. Finally, a small magenta ring around the probed texel, drawn in texture space, shows the student where the texel lies on the solid—and, on the cylinder, that the caps reuse the coordinates of the side.

```

< Mark the probed texel 81 > ≡
// Highlight the texel under the mouse in the inspector.
if (hoverOn > 0)
{
    float2 duv = abs(i.uv - hoverUV);
    float r = length(duv);
    if (r > 0.010 && r < 0.016) color = float3(1, 0.1, 0.8);
}

```

This code is used in section 72.

82. The *line shader* draws the arrows of the probe and the marker of the light. Each vertex carries its own colour, given in display (sRGB) values because that is how one chooses colours; the pixel shader converts it to linear light for the sRGB target.

```

< File shaders/gizmo.hlsl 82 > ≡
// Texture Explorer - colored lines (light marker, normal/tangent arrows).

cbuffer Frame : register(b0)
{
    row_major float4x4 world;
    row_major float4x4 viewProj;
};

struct VSIn { float3 pos : POSITION; float4 color : COLOR; };
struct VSOut { float4 svpos : SV_Position; float4 color : COLOR; };

VSOut VSLine(VSIn i)
{
    VSOut o;
    o.svpos = mul(float4(i.pos, 1), viewProj);
    o.color = i.color;
    return o;
}

float4 PSLine(VSOut i) : SV_Target
{
    return float4(pow(saturate(i.color.rgb), 2.2), i.color.a); // colors are given in
    ↪ display (sRGB) space
}

```

This code is written to the file `shaders/gizmo.hlsl`.

83. The renderer.

The renderer is the only part of the program that speaks Direct3D 11 at length. Its job is modest: create the device and the window's swap chain, compile the shaders, keep one GPU mesh per shape, and draw the scene into an off-screen image that the user interface then displays inside a panel. It knows nothing about the user interface, and the user interface knows nothing about Direct3D except the image it receives.

```

< File src/Renderer.h 83 >≡
#pragma once
#include "Material.h"
#include "Mesh.h"
#include <d3d11.h>
#include <dxgi1_2.h>
#include <DirectXMath.h>
#include <filesystem>
#include <string>
#include <vector>

< Constants of a frame, as seen from C++ 84 >

< Vertices of lines 85 >

class Renderer {
public:
    < Public interface of the renderer 86 >

private:
    < Private helpers of the renderer 87 >

    < Device and swap chain 88 >

    < The off-screen scene target 89 >

    < Pipeline objects 90 >
};

```

This code is written to the file `src/Renderer.h`.

84. This structure is the C++ image of the constant buffer of §66. Each line of four values is one 16-byte row of the buffer; the `static_assert` catches the most common mistake, a structure whose size is not a multiple of a row.

```

< Constants of a frame, as seen from C++ 84 >≡
// Mirrors cbuffer Frame in shaders/pbr.hlsl (16-byte aligned rows).
struct FrameConstants {
    DirectX::XMFLOAT4X4 world;
    DirectX::XMFLOAT4X4 viewProj;
    DirectX::XMFLOAT3 cameraPos; float bumpStrength;
    DirectX::XMFLOAT3 lightPos; float lightIntensity;
    DirectX::XMFLOAT3 lightColor; float dispScale;
    float dispMid, dispLod, metallic, ambient;
    int viewMode, useColor, useBump, useDisp;
    int useRough; float constRoughness; DirectX::XMFLOAT2 hoverUV;
    float hoverOn; DirectX::XMFLOAT3 pad;
};
static_assert(sizeof(FrameConstants) % 16 == 0);

```

This code is used in section 83.

85. Lines are given as pairs of vertices with a colour each.

```

< Vertices of lines 85 >≡
struct LineVertex {
    DirectX::XMFLOAT3 pos;
    DirectX::XMFLOAT4 color;
};

```

This code is used in section 83.

86. The public interface follows the life of a frame: `BeginFrame` prepares the window for the user interface, `RenderScene` draws the 3D view off-screen, `Present` shows the result. The number of segments per patch is public because the user interface needs it to choose the mipmap level for displacement.

```

< Public interface of the renderer 86 >≡
static constexpr int kMeshSegments = 256;

bool Init(HWND hwnd);
void Shutdown();
void Resize(UINT width, UINT height);
void BeginFrame(const float clear[4]);
void Present(bool vsync);

bool ReloadShaders(const std::filesystem::path& dir, std::string& log);

// Renders the 3D scene into an offscreen target of the given size and returns
// a view of the result that ImGui can display.
ID3D11ShaderResourceView* RenderScene(UINT width, UINT height, const FrameConstants& fc,
                                     const Material& material, Shape shape,
                                     const std::vector<LineVertex>& lines);

ID3D11Device* Device() const { return device_.Get(); }
ID3D11DeviceContext* Context() const { return ctx_.Get(); }
This code is used in section 83.

```

87.

```

< Private helpers of the renderer 87 >≡
struct GpuMesh {
    ComPtr<ID3D11Buffer> vb, ib;
    UINT indexCount = 0;
};
void CreateBackbufferView();
void EnsureSceneTarget(UINT width, UINT height);
GpuMesh& MeshFor(Shape shape);
This code is used in section 83.

```

88. All Direct3D objects are held by ComPtr, a smart pointer that releases them automatically; the renderer has no destructor to write.

```

< Device and swap chain 88 >≡
ComPtr<ID3D11Device> device_;
ComPtr<ID3D11DeviceContext> ctx_;
ComPtr<IDXGISwapChain1> swapChain_;
ComPtr<ID3D11RenderTargetView> backbufferRtv_;
This code is used in section 83.

```

89.

```

< The off-screen scene target 89 >≡
// Offscreen scene target: 4x MSAA color + depth, resolved to a single-sample texture.
UINT sceneW_ = 0, sceneH_ = 0;
ComPtr<ID3D11Texture2D> msaaColor_, resolved_, depth_;
ComPtr<ID3D11RenderTargetView> msaaRtv_;
ComPtr<ID3D11DepthStencilView> dsv_;
ComPtr<ID3D11ShaderResourceView> resolvedSrv_;
This code is used in section 83.

```

90.

```

< Pipeline objects 90 >≡
ComPtr<ID3D11VertexShader> meshVs_, lineVs_;
ComPtr<ID3D11PixelShader> meshPs_, linePs_;
ComPtr<ID3D11InputLayout> meshLayout_, lineLayout_;
ComPtr<ID3D11Buffer> frameCb_, lineVb_;
ComPtr<ID3D11SamplerState> sampler_;
ComPtr<ID3D11RasterizerState> rasterNoCull_;
ComPtr<ID3D11DepthStencilState> depthOn_, depthOff_;
ComPtr<ID3D11BlendState> alphaBlend_;
ComPtr<ID3D11ShaderResourceView> fallbackGray_;
GpuMesh meshes_[int(Shape::Count)];
static constexpr UINT kMaxLineVertices = 4096;
This code is used in section 83.

```

91. The implementation file is a sequence of member functions.

```

< File src/Renderer.cpp 91 > ≡
#include "Renderer.h"
#include <d3dcompiler.h>
#include <algorithm>
#include <cstring>

```

```
using namespace DirectX;
```

```

< Functions of the renderer 92 >

```

This code is written to the file `src/Renderer.cpp`.

92. *Initialization* creates, once and for all, everything that does not depend on the size of the window or on the shaders.

We ask for a hardware device of feature level 11. In debug builds we also ask for the debug layer, which reports misuses of the API in the debugger's output; but the debug layer is an optional component of Windows, so if it is missing we try again without it. (This piece stays inline, rather than in a module of its own, because the preprocessor lines must start in the first column of the file.)

```

< Functions of the renderer 92 > ≡
bool Renderer::Init(HWND hwnd) {
    UINT flags = 0;
#ifdef _DEBUG
    flags |= D3D11_CREATE_DEVICE_DEBUG;
#endif
    const D3D_FEATURE_LEVEL levels[] = {D3D_FEATURE_LEVEL_11_1, D3D_FEATURE_LEVEL_11_0};
    HRESULT hr = D3D11CreateDevice(nullptr, D3D_DRIVER_TYPE_HARDWARE, nullptr, flags,
        ↪levels, 2,
        D3D11_SDK_VERSION, &device_, nullptr, &ctx_);
    if (FAILED(hr) && (flags & D3D11_CREATE_DEVICE_DEBUG)) { // debug layer not installed
        hr = D3D11CreateDevice(nullptr, D3D_DRIVER_TYPE_HARDWARE, nullptr, 0, levels, 2,
            D3D11_SDK_VERSION, &device_, nullptr, &ctx_);
    }
    if (FAILED(hr)) return false;

    < Create the swap chain 93 >

    < Create the sampler 94 >

    < Create the rasterizer state 95 >

    < Create the depth states 96 >

    < Create the blend state for lines 97 >

    < Create the constant buffer 98 >

    < Create the line buffer 99 >

    < Create a grey texture for missing maps 100 >
    return true;
}

```

See also sections 101, 102, 103, 106, 110 and 111. This code is used in section 91.

93. The swap chain is the pair of images that alternate on the screen. We use the modern *flip* model, which requires a non-sRGB format for the buffers; that suits Dear ImGui, whose colours are already display values. Alt+Enter, which would switch to an exclusive full-screen mode, is disabled.

```

< Create the swap chain 93 > ≡
ComPtr<IDXGIDevice> dxgiDevice;
ComPtr<IDXGIAdapter> adapter;
ComPtr<IDXGIFactory2> factory;
device_.As(&dxgiDevice);
dxgiDevice->GetAdapter(&adapter);
adapter->GetParent(IID_PPV_ARGS(&factory));

DXGI_SWAP_CHAIN_DESC1 sd{};
sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;

```

```

sd.SampleDesc.Count = 1;
sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
sd.BufferCount = 2;
sd.SwapEffect = DXGI_SWAP_EFFECT_FLIP_DISCARD;
if (FAILED(factory->CreateSwapChainForHwnd(device_.Get(), hwnd, &sd, nullptr, nullptr,
    &swapChain_)))
    return false;
factory->MakeWindowAssociation(hwnd, DXGI_MWA_NO_ALT_ENTER);
CreateBackbufferView();

```

This code is used in section 92.

94. The *sampler* decides how texels become a colour. Anisotropic filtering keeps the texture sharp on surfaces seen at grazing angles, which on a sphere is everywhere near the silhouette. Wrap addressing makes the textures repeat, as they were designed to.

```

< Create the sampler 94 > ≡
// Fixed pipeline state.
D3D11_SAMPLER_DESC samp{};
samp.Filter = D3D11_FILTER_ANISOTROPIC;
samp.MaxAnisotropy = 8;
samp.AddressU = samp.AddressV = samp.AddressW = D3D11_TEXTURE_ADDRESS_WRAP;
samp.MaxLOD = D3D11_FLOAT32_MAX;
device_->CreateSamplerState(&samp, &sampler_);

```

This code is used in section 92.

95. We do not cull back faces. The patches were designed for their texture coordinates, not for a consistent winding order of their triangles, and drawing both sides also behaves sensibly when a student flies the camera into a solid.

```

< Create the rasterizer state 95 > ≡
D3D11_RASTERIZER_DESC rs{};
rs.FillMode = D3D11_FILL_SOLID;
rs.CullMode = D3D11_CULL_NONE;
rs.DepthClipEnable = TRUE;
rs.MultisampleEnable = TRUE;
rs.AntialiasedLineEnable = FALSE;
device_->CreateRasterizerState(&rs, &rasterNoCull_);

```

This code is used in section 92.

96. Two depth states: the normal one, and one that ignores depth, used to draw the arrows of the probe faintly even when the solid hides them.

```

< Create the depth states 96 > ≡
D3D11_DEPTH_STENCIL_DESC ds{};
ds.DepthEnable = TRUE;
ds.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ALL;
ds.DepthFunc = D3D11_COMPARISON_LESS_EQUAL;
device_->CreateDepthStencilState(&ds, &depthOn_);
ds.DepthEnable = FALSE;
ds.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ZERO;
device_->CreateDepthStencilState(&ds, &depthOff_);

```

This code is used in section 92.

97. Ordinary “over” blending, $c = \alpha c_{src} + (1 - \alpha) c_{dst}$, for those faint lines.

```

< Create the blend state for lines 97 > ≡
D3D11_BLEND_DESC bd{};
auto& rt = bd.RenderTarget[0];
rt.BlendEnable = TRUE;
rt.SrcBlend = D3D11_BLEND_SRC_ALPHA;
rt.DestBlend = D3D11_BLEND_INV_SRC_ALPHA;
rt.BlendOp = D3D11_BLEND_OP_ADD;
rt.SrcBlendAlpha = D3D11_BLEND_ONE;
rt.DestBlendAlpha = D3D11_BLEND_ZERO;
rt.BlendOpAlpha = D3D11_BLEND_OP_ADD;
rt.RenderTargetWriteMask = D3D11_COLOR_WRITE_ENABLE_ALL;
device_->CreateBlendState(&bd, &alphaBlend_);

```

This code is used in section 92.

98. Buffers that change every frame are *dynamic*: the CPU writes them, the GPU only reads them.

```

< Create the constant buffer 98 > ≡
D3D11_BUFFER_DESC cb{};
cb.ByteWidth = sizeof(FrameConstants);
cb.Usage = D3D11_USAGE_DYNAMIC;
cb.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
cb.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;
device_>CreateBuffer(&cb, nullptr, &frameCb_);
This code is used in section 92.

```

99.

```

< Create the line buffer 99 > ≡
D3D11_BUFFER_DESC lb{};
lb.ByteWidth = sizeof(LineVertex) * kMaxLineVertices;
lb.Usage = D3D11_USAGE_DYNAMIC;
lb.BindFlags = D3D11_BIND_VERTEX_BUFFER;
lb.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;
device_>CreateBuffer(&lb, nullptr, &lineVb_);
This code is used in section 92.

```

100. A shader that samples an empty slot reads zeros, which for a height map means “very deep” and for a roughness map “mirror”. A neutral grey texture is a safer stand-in.

```

< Create a grey texture for missing maps 100 > ≡
// 1x1 mid-gray texture bound when a material lacks a map.
const uint32_t gray = 0xff808080;
D3D11_TEXTURE2D_DESC td{};
td.Width = td.Height = 1;
td.MipLevels = td.ArraySize = 1;
td.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
td.SampleDesc.Count = 1;
td.BindFlags = D3D11_BIND_SHADER_RESOURCE;
D3D11_SUBRESOURCE_DATA init{&gray, 4, 0};
ComPtr<ID3D11Texture2D> tex;
device_>CreateTexture2D(&td, &init, &tex);
device_>CreateShaderResourceView(tex.Get(), nullptr, &fallbackGray_);
This code is used in section 92.

```

101. The remaining housekeeping of the window: shutting down, creating a view of the back buffer, resizing the swap chain when the window changes size (all views of the old buffers must be released first), and beginning and presenting a frame. Presenting with `vsync` synchronizes the program with the display, so it does not waste the GPU.

```

< Functions of the renderer 92 > +≡
void Renderer::Shutdown() {
    if (ctx_) ctx_>ClearState();
}

void Renderer::CreateBackbufferView() {
    ComPtr<ID3D11Texture2D> bb;
    swapChain_>GetBuffer(0, IID_PPV_ARGS(&bb));
    device_>CreateRenderTargetView(bb.Get(), nullptr, &backbufferRtv_);
}

void Renderer::Resize(UINT width, UINT height) {
    if (!swapChain_ || width == 0 || height == 0) return;
    backbufferRtv_.Reset();
    ctx_>OMSetRenderTargets(0, nullptr, nullptr);
    swapChain_>ResizeBuffers(0, width, height, DXGI_FORMAT_UNKNOWN, 0);
    CreateBackbufferView();
}

void Renderer::BeginFrame(const float clear[4]) {
    ctx_>OMSetRenderTargets(1, backbufferRtv_.GetAddressOf(), nullptr);
    ctx_>ClearRenderTargetView(backbufferRtv_.Get(), clear);
}

void Renderer::Present(bool vsync) { swapChain_>Present(vsync ? 1 : 0, 0); }

```

102. *Compiling shaders.* The shaders are compiled from their source files at run time, which costs a fraction of a second and buys the possibility of editing them while the program runs. Messages of the compiler—errors and warnings—are collected in `log` so the interface can show them.

```

<Functions of the renderer 92> +=
static ComPtr<ID3DBlob> Compile(const std::filesystem::path& file, const char* entry, const
    ↪char* target,
                                std::string& log) {
    UINT flags = D3DCOMPILER_ENABLE_STRICTNESS;
#ifdef _DEBUG
    flags |= D3DCOMPILER_DEBUG | D3DCOMPILER_SKIP_OPTIMIZATION;
#endif
    ComPtr<ID3DBlob> code, errors;
    const HRESULT hr = D3DCompileFromFile(file.c_str(), nullptr,
        ↪D3D_COMPILE_STANDARD_FILE_INCLUDE, entry,
                                target, flags, 0, &code, &errors);
    if (errors) log += static_cast<const char*>(errors->GetBufferPointer());
    if (FAILED(hr)) {
        if (!errors) log += "Cannot compile " + file.string() + "\n";
        return nullptr;
    }
    return code;
}

```

103. Reloading is transactional: the new shaders replace the old ones only if all four compile and all objects can be created. A typing mistake in the shader therefore leaves the program running with the previous version, and the error message on the screen.

```

<Functions of the renderer 92> +=
bool Renderer::ReloadShaders(const std::filesystem::path& dir, std::string& log) {
    log.clear();
    auto vs = Compile(dir / "pbr.hlsl", "VSMain", "vs_5_0", log);
    auto ps = Compile(dir / "pbr.hlsl", "PSMain", "ps_5_0", log);
    auto lvs = Compile(dir / "gizmo.hlsl", "VSLine", "vs_5_0", log);
    auto lps = Compile(dir / "gizmo.hlsl", "PSLine", "ps_5_0", log);
    if (!vs || !ps || !lvs || !lps) return false; // keep the previous working shaders

    <Describe the two vertex layouts 104>
    <Create the shader objects, keeping the old ones on failure 105>
    return true;
}

```

104. An *input layout* tells the GPU how to read the fields of a vertex from the buffer; the names (`POSITION`, `NORMAL`, ...) match the semantics in the shaders' input structures, and `offsetof` keeps the C++ structures the only source of truth.

```

<Describe the two vertex layouts 104> =
const D3D11_INPUT_ELEMENT_DESC meshElems[] = {
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(Vertex, pos),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(Vertex, normal),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TANGENT", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, offsetof(Vertex, tangent),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, offsetof(Vertex, uv),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
};

const D3D11_INPUT_ELEMENT_DESC lineElems[] = {
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(LineVertex, pos),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, offsetof(LineVertex, color),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
};

```

This code is used in section 103.

105.

```

<Create the shader objects, keeping the old ones on failure 105> =
ComPtr<ID3D11VertexShader> newVs, newLineVs;
ComPtr<ID3D11PixelShader> newPs, newLinePs;

```

```

ComPtr<ID3D11InputLayout> newLayout, newLineLayout;
if (FAILED(device_>CreateVertexShader(vs->GetBufferPointer(), vs->GetBufferSize(), nullptr,
    ↪&newVs)) ||
    FAILED(device_>CreatePixelShader(ps->GetBufferPointer(), ps->GetBufferSize(), nullptr,
    ↪&newPs)) ||
    FAILED(device_>CreateVertexShader(lvs->GetBufferPointer(), lvs->GetBufferSize(),
    ↪nullptr, &newLineVs)) ||
    FAILED(device_>CreatePixelShader(lps->GetBufferPointer(), lps->GetBufferSize(),
    ↪nullptr, &newLinePs)) ||
    FAILED(device_>CreateInputLayout(meshElems, 4, vs->GetBufferPointer(),
    ↪vs->GetBufferSize(), &newLayout)) ||
    FAILED(device_>CreateInputLayout(lineElems, 2, lvs->GetBufferPointer(),
    ↪lvs->GetBufferSize(), &newLineLayout))) {
    log += "Failed to create shader objects\n";
    return false;
}
meshVs_ = newVs; meshPs_ = newPs; lineVs_ = newLineVs; linePs_ = newLinePs;
meshLayout_ = newLayout; lineLayout_ = newLineLayout;
This code is used in section 103.

```

106. The *off-screen target* is recreated whenever the viewport panel changes size. It is drawn with four samples per pixel (multisample antialiasing), which smooths the silhouette of the solid and the thin lines of the probe, and then *resolved* into an ordinary texture that Dear ImGui can display.

```

< Functions of the renderer 92 > +=
void Renderer::EnsureSceneTarget(UINT width, UINT height) {
    if (width == sceneW_ && height == sceneH_ && msaaColor_) return;
    sceneW_ = width;
    sceneH_ = height;
    < Create the multisampled colour target 107 >

    < Create the resolved image 108 >

    < Create the depth buffer 109 >
}

```

107. The same trick as in the texture loader (*< Create a texture with a full mipmap chain 45 >*): a typeless texture, written through an sRGB view so that the GPU encodes our linear colours for display, and read through a plain view so that Dear ImGui copies the encoded bytes to the screen unchanged.

```

< Create the multisampled colour target 107 > ≡
// Typeless so it can be rendered as sRGB (gamma-correct lighting) but shown by
// ImGui as UNORM, i.e. the already-encoded values reach the screen untouched.
D3D11_TEXTURE2D_DESC td{};
td.Width = width;
td.Height = height;
td.MipLevels = td.ArraySize = 1;
td.Format = DXGI_FORMAT_R8G8B8A8_TYPELESS;
td.SampleDesc.Count = 4;
td.BindFlags = D3D11_BIND_RENDER_TARGET;
device_>CreateTexture2D(&td, nullptr, &msaaColor_);
D3D11_RENDER_TARGET_VIEW_DESC rd{};
rd.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
rd.ViewDimension = D3D11_RTV_DIMENSION_TEXTURE2DMS;
device_>CreateRenderTargetView(msaaColor_.Get(), &rd, &msaaRtv_);
This code is used in section 106.

```

108.

```

< Create the resolved image 108 > ≡
td.SampleDesc.Count = 1;
td.BindFlags = D3D11_BIND_SHADER_RESOURCE;
device_>CreateTexture2D(&td, nullptr, &resolved_);
D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
sd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
sd.Texture2D.MipLevels = 1;
device_>CreateShaderResourceView(resolved_.Get(), &sd, &resolvedSrv_);
This code is used in section 106.

```

109.

```

< Create the depth buffer 109 > ≡
td.Format = DXGI_FORMAT_D32_FLOAT;
td.SampleDesc.Count = 4;
td.BindFlags = D3D11_BIND_DEPTH_STENCIL;
device_>CreateTexture2D(&td, nullptr, &depth_);
device_>CreateDepthStencilView(depth_.Get(), nullptr, &dsv_);
This code is used in section 106.

```

110. *GPU meshes* are built the first time a shape is shown and kept afterwards. They never change, so they are IMMUTABLE, the fastest kind of buffer.

```

< Functions of the renderer 92 > +≡
Renderer::GpuMesh& Renderer::MeshFor(Shape shape) {
    GpuMesh& m = meshes_[int(shape)];
    if (m.vb) return m;
    const MeshData data = BuildMesh(shape, kMeshSegments);
    D3D11_BUFFER_DESC bd{};
    bd.Usage = D3D11_USAGE_IMMUTABLE;
    bd.ByteWidth = UINT(data.vertices.size() * sizeof(Vertex));
    bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;
    D3D11_SUBRESOURCE_DATA init{data.vertices.data(), 0, 0};
    device_>CreateBuffer(&bd, &init, &m.vb);
    bd.ByteWidth = UINT(data.indices.size() * sizeof(uint32_t));
    bd.BindFlags = D3D11_BIND_INDEX_BUFFER;
    init.pSysMem = data.indices.data();
    device_>CreateBuffer(&bd, &init, &m.ib);
    m.indexCount = UINT(data.indices.size());
    return m;
}

```

111. *Drawing the scene*, at last. The function returns nullptr when there is nothing to draw, for instance when the shaders failed to compile the first time.

```

< Functions of the renderer 92 > +≡
ID3D11ShaderResourceView* Renderer::RenderScene(UINT width, UINT height, const
    ↳ FrameConstants& fc,
                                const Material& material, Shape shape,
                                const std::vector<LineVertex>& lines) {
    if (width == 0 || height == 0 || !meshVs_) return nullptr;
    EnsureSceneTarget(width, height);

    < Clear the scene target and set the pipeline state 112 >

    < Upload the frame constants 113 >

    < Draw the surface 114 >

    < Draw the lines 115 >

    < Unbind the textures and resolve the multisampled image 116 >
    return resolvedSrv_.Get();
}

```

112. The background colour is given in linear values, because the target is sRGB; 0.035 becomes about 0.2 on the screen.

```

< Clear the scene target and set the pipeline state 112 > ≡
const float clear[4] = {0.035f, 0.04f, 0.05f, 1}; // linear (sRGB target)
ctx_>ClearRenderTargetView(msaaRtv_.Get(), clear);
ctx_>ClearDepthStencilView(dsv_.Get(), D3D11_CLEAR_DEPTH, 1, 0);
ctx_>OMSetRenderTargets(1, msaaRtv_.GetAddressOf(), dsv_.Get());
const D3D11_VIEWPORT vp{0, 0, float(width), float(height), 0, 1};
ctx_>RSSetViewports(1, &vp);
ctx_>RSSetState(rasterNoCull_.Get());
ctx_>OMSetDepthStencilState(depthOn_.Get(), 0);
ctx_>OMSetBlendState(nullptr, nullptr, 0xffffffff);
This code is used in section 111.

```

113. `WRITE_DISCARD` gives us fresh memory for the buffer, so the CPU never waits for the GPU to finish with last frame's constants.

```
< Upload the frame constants 113 > ≡
D3D11_MAPPED_SUBRESOURCE ms;
ctx_>Map(frameCb_.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &ms);
memcpy(ms.pData, &fc, sizeof(fc));
ctx_>Unmap(frameCb_.Get(), 0);
```

This code is used in section 111.

114. The surface is one indexed draw call. The four maps are bound to both the vertex shader (which needs the displacement map) and the pixel shader; the colour map through its sRGB view, the data maps through their linear views.

```
< Draw the surface 114 > ≡
// Surface
GpuMesh& mesh = MeshFor(shape);
const UINT stride = sizeof(Vertex), offset = 0;
ctx_>IASetInputLayout(meshLayout_.Get());
ctx_>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
ctx_>IASetVertexBuffers(0, 1, mesh.vb.GetAddressOf(), &stride, &offset);
ctx_>IASetIndexBuffer(mesh.ib.Get(), DXGI_FORMAT_R32_UINT, 0);
ctx_>VSSetShader(meshVs_.Get(), nullptr, 0);
ctx_>PSSetShader(meshPs_.Get(), nullptr, 0);
ctx_>VSSetConstantBuffers(0, 1, frameCb_.GetAddressOf());
ctx_>PSSetConstantBuffers(0, 1, frameCb_.GetAddressOf());
auto srv = [&](MapKind k, bool srgb) -> ID3D11ShaderResourceView* {
    const auto& map = material.maps[k];
    if (!map) return fallbackGray_.Get();
    return srgb ? map->srvSrgb.Get() : map->srvLinear.Get();
};
ID3D11ShaderResourceView* srvs[4] = {srv(MapColor, true), srv(MapBump, false),
    ↪srv(MapDisplacement, false),
    ↪srv(MapRoughness, false)};
ctx_>VSSetShaderResources(0, 4, srvs);
ctx_>PSSetShaderResources(0, 4, srvs);
ctx_>VSSetSamplers(0, 1, sampler_.GetAddressOf());
ctx_>PSSetSamplers(0, 1, sampler_.GetAddressOf());
ctx_>DrawIndexed(mesh.indexCount, 0, 0);
```

This code is used in section 111.

115. The lines are drawn twice from the same buffer: the first half holds copies with 30% of their opacity, drawn without depth test, so that hidden parts of an arrow remain faintly visible through the solid; the second half holds the originals, drawn with the depth test. The line shader uses the same constant buffer, of which it only reads the first two matrices.

```
< Draw the lines 115 > ≡
// Lines: a faint "x-ray" pass ignoring depth, then a depth-tested pass on top.
if (!lines.empty()) {
    const UINT n = UINT(std::min<size_t>(lines.size(), kMaxLineVertices / 2));
    ctx_>Map(lineVb_.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &ms);
    auto* dst = static_cast<LineVertex*>(ms.pData);
    for (UINT i = 0; i < n; ++i) {
        dst[i] = lines[i];
        dst[i].color.w *= 0.3f;
        dst[n + i] = lines[i];
    }
    ctx_>Unmap(lineVb_.Get(), 0);
    const UINT lstride = sizeof(LineVertex);
    ctx_>IASetInputLayout(lineLayout_.Get());
    ctx_>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_LINELIST);
    ctx_>IASetVertexBuffers(0, 1, lineVb_.GetAddressOf(), &lstride, &offset);
    ctx_>VSSetShader(lineVs_.Get(), nullptr, 0);
    ctx_>PSSetShader(linePs_.Get(), nullptr, 0);
    ctx_>OMSetBlendState(alphaBlend_.Get(), nullptr, 0xffffffff);
    ctx_>OMSetDepthStencilState(depthOff_.Get(), 0);
    ctx_>Draw(n, 0);
    ctx_>OMSetDepthStencilState(depthOn_.Get(), 0);
```

```
    ctx_->Draw(n, n);  
}
```

This code is used in section 111.

116. The textures are unbound before the next frame, in which some of them may be written (the mipmap generation of a newly loaded material, for instance); Direct3D refuses to have a resource bound for reading and writing at once. The resolve averages the four samples of each pixel, interpreting them as sRGB so that the average is taken in linear light.

```
< Unbind the textures and resolve the multisampled image 116 > ≡  
ID3D11ShaderResourceView* nullSrvs[4] = {};  
ctx_->VSSetShaderResources(0, 4, nullSrvs);  
ctx_->PSSetShaderResources(0, 4, nullSrvs);  
ctx_->ResolveSubresource(resolved_.Get(), 0, msaaColor_.Get(), 0,  
    ⇨ DXGI_FORMAT_R8G8B8A8_UNORM_SRGB);
```

This code is used in section 111.

117. The application and its user interface.

The class **App** owns the renderer and the material library, holds the state that the user can change, and builds the user interface every frame. The interface is made with Dear ImGui, an *immediate-mode* library: there are no widget objects to create and destroy; every frame, the program simply calls `ImGui::Button`, `ImGui::SliderFloat` and so on, and each call both draws the widget and reports whether the user interacted with it. This style fits a teaching tool very well, because the state lives in plain variables that the rest of the program reads directly.

```

< File src/App.h 117 >≡
#pragma once
#include "Inspector.h"
#include "Material.h"
#include "Mesh.h"
#include "Renderer.h"
#include <filesystem>
#include <string>

struct ImVec2;

< The orbiting camera 118 >

< The orbiting light 119 >

class App {
public:
    < Public interface of the application 120 >

private:
    < Private member functions of the application 121 >

    < Resources of the application 122 >

    < State of the scene 123 >

    < State of the shading 124 >

    < State of the inspector 125 >
};

```

This code is written to the file `src/App.h`.

118. The camera always looks at the origin from a point on a sphere, given by spherical coordinates: *yaw* (rotation around the vertical axis), *pitch* (elevation) and *distance*. The default yaw puts the camera on the side of the solid where the middle of the texture, $u = \frac{1}{2}$, is mapped.

```

< The orbiting camera 118 >≡
struct OrbitCamera {
    float yaw = -1.3f, pitch = 0.3f, distance = 3.6f; // faces the texture's center (u =
    ↪ 0.5)
    DirectX::XMFLLOAT3 Eye() const;
};

```

This code is used in section 117.

119. The light is a point light that moves in the same way, plus an intensity, a colour, and an optional automatic orbit around the vertical axis.

```

< The orbiting light 119 >≡
struct OrbitLight {
    float azimuth = -2.0f, elevation = 0.5f, radius = 2.4f;
    float intensity = 5.0f;
    float color[3] = {1.0f, 0.97f, 0.92f};
    bool autoOrbit = false;
    float speed = 0.6f; // rad/s
    DirectX::XMFLLOAT3 Position() const;
};

```

This code is used in section 117.

120. The main program needs only to initialize the application, call `Frame` once per frame, and reach the renderer to set up Dear ImGui and to present.

```

<Public interface of the application 120>≡
bool Init(HWND hwnd);
void Shutdown();
void Frame(float dt);           // builds the UI and renders the scene
Renderer& GetRenderer() { return renderer_; }
void ReloadShaders();
This code is used in section 117.

```

121. One private function per panel of the interface.

```

<Private member functions of the application 121>≡
void BuildDockLayout(unsigned int dockspaceId);
void UiMaterials();
void UiScene();
void UiMaps();
void UiInspector();
void UiViewport();
void UiConcepts();
// Draws a map image; if hovered, records the hovered uv. Click pins the probe.
void MapImage(MapKind kind, const ImVec2& size);
Material& Current() { return library_.Items()[current_]; }
DirectX::XMATRIX World() const;
This code is used in section 117.

```

122.

```

<Resources of the application 122>≡
Renderer renderer_;
MaterialLibrary library_;
std::filesystem::path root_;
std::string loadError_, shaderLog_;
bool shadersOk_ = false;
This code is used in section 117.

```

123. The state of the scene: which material and shape are shown, the camera, the light, and the rotation of the solid around its vertical axis.

```

<State of the scene 123>≡
int current_ = 0;
Shape shape_ = Shape::Sphere;
OrbitCamera camera_;
OrbitLight light_;
bool rotate_ = false;
float rotSpeed_ = 0.5f, rotAngle_ = 0;
This code is used in section 117.

```

124.

```

<State of the shading 124>≡
// Shading
BumpSettings bump_;
bool useColor_ = true, useBump_ = true, useRough_ = true;
float constRough_ = 0.5f, metallic_ = 0, ambient_ = 0.03f;
int viewMode_ = 0;
bool showArrows_ = true;
This code is used in section 117.

```

125. The state of the probe deserves attention. The probe follows the mouse while it hovers any map; a left click *pins* it, so that the user can then move the probe elsewhere—to rotate the camera, say—while the readouts stay. The flag `hoveredThisFrame_` is cleared at the start of each frame and set by any map image that the mouse is over; at the end of the interface, an unpinned probe that nobody hovered becomes invalid.

```

<State of the inspector 125>≡
// Inspector
MapKind inspected_ = MapColor;
bool hoveredThisFrame_ = false, pinned_ = false;
DirectX::XMFLOAT2 probeUV_{0.5f, 0.5f};
bool probeValid_ = false;
HoverInfo info_;
This code is used in section 117.

```

126. The implementation begins with a few helpers of local interest, followed by the member functions.

```

<File src/App.cpp 126>≡
#include "App.h"
#include <imgui.h>
#include <imgui_internal.h> // DockBuilder
#include <shellapi.h>
#include <algorithm>
#include <cmath>
#include <cstdio>

```

```
using namespace DirectX;
```

```

namespace {
<Private helpers of the application 127>
} // namespace

```

```
<Functions of the application 136>
```

This code is written to the file `src/App.cpp`.

127.

```

<Private helpers of the application 127>≡
constexpr float kPi = 3.14159265f;

```

See also sections 128, 129, 130, 131, 132, 133, 134 and 135. This code is used in section 126.

128. Dear ImGui refers to textures by an opaque number; with the Direct3D 11 back end, that number is the address of a shader resource view.

```

<Private helpers of the application 127> +≡
ImTextureID Tex(ID3D11ShaderResourceView* srv) { return (ImTextureID)(intptr_t)srv; }

```

129. The program needs its `shaders` and `assets` folders. It looks for them in the folder of the executable and in its ancestors, which covers both a build tree (`build/Release/`) and a copied installation; as a last resort it uses the source folder recorded by CMake at compile time.

```

<Private helpers of the application 127> +≡
std::filesystem::path FindRoot() {
    wchar_t buf[MAX_PATH];
    GetModuleFileNameW(nullptr, buf, MAX_PATH);
    for (auto p = std::filesystem::path(buf).parent_path(); p.has_relative_path(); p =
        ↪p.parent_path())
        if (std::filesystem::exists(p / "shaders" / "pbr.hlsl")) return p;
    return std::filesystem::path(TEXEX_ROOT);
}

```

130. Rows of the inspector's tables. A vector is shown with its three components and, when it is a unit vector, a swatch of the colour that encodes it by $\frac{1}{2}\mathbf{n} + \frac{1}{2}$ (§6). Hovering the label explains the row.

```

<Private helpers of the application 127> +≡
ImVec4 Encode(const XMFL0AT3& n) { return {n.x * 0.5f + 0.5f, n.y * 0.5f + 0.5f, n.z * 0.5f
    ↪+ 0.5f, 1}; }

```

131.

```

<Private helpers of the application 127> +≡
void Vec3Row(const char* label, const XMFL0AT3& v, bool swatch, const char* tip) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    ImGui::TextUnformatted(label);
    if (tip && ImGui::IsItemHovered()) ImGui::SetTooltip("%s", tip);
    ImGui::TableNextColumn();
    ImGui::Text("(%.3f, %.3f, %.3f)", v.x, v.y, v.z);
    ImGui::TableNextColumn();
    if (swatch) {
        ImGui::PushID(label);
        ImGui::ColorButton("##sw", Encode(v), ImGuiColorEditFlags_NoTooltip, ImVec2(36, 0));
        ImGui::PopID();
    }
}

```

132. A scalar is shown both as a fraction and as the byte actually stored in the image, because students often meet both forms.

```

<Private helpers of the application 127> +≡
void ScalarRow(const char* label, float v, const char* tip) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    ImGui::TextUnformatted(label);
    if (tip && ImGui::IsItemHovered()) ImGui::SetTooltip("%s", tip);
    ImGui::TableNextColumn();
    ImGui::Text("%.4f    (%d/255)", v, int(std::lround(v * 255)));
    ImGui::TableNextColumn();
    ImGui::ColorButton("##g", ImVec4(v, v, v, 1), ImGuiColorEditFlags_NoTooltip, ImVec2(36,
        ↪0));
}

```

133. An arrow is a segment plus two short segments for the head. The head lies in the plane spanned by the arrow and the vertical (or the x axis, if the arrow is vertical).

```

<Private helpers of the application 127> +≡
void AddArrow(std::vector<LineVertex>& out, XMVECTOR from, XMVECTOR dir, float len, XMFLOAT4
    ↪color) {
    const XMVECTOR to = from + XMVector3Normalize(dir) * len;
    XMFLOAT3 a, b;
    XMStoreFloat3(&a, from);
    XMStoreFloat3(&b, to);
    out.push_back({a, color});
    out.push_back({b, color});
    // Small arrow head: two short lines in a plane containing dir.
    XMVECTOR side = XMVector3Cross(dir, XMVectorSet(0, 1, 0, 0));
    if (XMVectorGetX(XMVector3LengthSq(side)) < 1e-6f) side = XMVector3Cross(dir,
        ↪XMVectorSet(1, 0, 0, 0));
    side = XMVector3Normalize(side) * (len * 0.12f);
    const XMVECTOR back = to - XMVector3Normalize(dir) * (len * 0.2f);
    XMFLOAT3 h1, h2;
    XMStoreFloat3(&h1, back + side);
    XMStoreFloat3(&h2, back - side);
    out.push_back({b, color}); out.push_back({h1, color});
    out.push_back({b, color}); out.push_back({h2, color});
}

```

134. A circle of radius r in the plane spanned by two orthonormal vectors $\mathbf{a}_x$, $\mathbf{a}_y$, approximated by 24 segments. Three of them, in the three coordinate planes, make the marker of the light.

```

<Private helpers of the application 127> +≡
void AddCircle(std::vector<LineVertex>& out, XMFLOAT3 c, XMVECTOR ax, XMVECTOR ay, float r,
    ↪XMFLOAT4 color) {
    const XMVECTOR center = XMLoadFloat3(&c);
    const int n = 24;
    for (int i = 0; i < n; ++i) {
        const float a0 = 2 * kPi * i / n, a1 = 2 * kPi * (i + 1) / n;
        XMFLOAT3 p0, p1;
        XMStoreFloat3(&p0, center + r * (std::cos(a0) * ax + std::sin(a0) * ay));
        XMStoreFloat3(&p1, center + r * (std::cos(a1) * ax + std::sin(a1) * ay));
        out.push_back({p0, color});
        out.push_back({p1, color});
    }
}

```

135. The names of the debug views, in the order of the `viewMode` values of *<Choose a debug view 76>*.

```

<Private helpers of the application 127> +≡
const char* kViewModes[] = {"Lit (GGX)", "Albedo only", "World normal (bumped)",
    ↪"Tangent-space normal",
    "Roughness", "Bump height", "UV coordinates"};

```

136. The positions of the camera and of the light follow from their spherical coordinates (§118): with elevation ϕ and azimuth θ , the point at distance d is $d(\cos \phi \sin \theta, \sin \phi, \cos \phi \cos \theta)$.

```

<Functions of the application 136> ≡
XMFLOAT3 OrbitCamera::Eye() const {
    return {distance * std::cos(pitch) * std::sin(yaw), distance * std::sin(pitch),
        distance * std::cos(pitch) * std::cos(yaw)};
}

```

```
}
```

See also sections 137, 138, 139, 140, 141, 142, 147, 148, 151, 157, 160, 161, 166 and 173. This code is used in section 126.

137.

```
< Functions of the application 136 > +=
XMFLOAT3 OrbitLight::Position() const {
    return {radius * std::cos(elevation) * std::sin(azimuth), radius * std::sin(elevation),
            radius * std::cos(elevation) * std::cos(azimuth)};
}
```

138. *Initialization.* All ten materials are loaded at once, about a second's work, so that the list can show a thumbnail of each.

```
< Functions of the application 136 > +=
bool App::Init(HWND hwnd) {
    if (!renderer_.Init(hwnd)) return false;
    root_ = FindRoot();
    if (library_.Load(root_ / "assets" / "materials", loadError_))
        for (int i = 0; i < int(library_.Items().size()); ++i) // all, for the thumbnails
            library_.EnsureLoaded(i, renderer_.Device(), renderer_.Context());
    ReloadShaders();
    return true;
}
```

139.

```
< Functions of the application 136 > +=
void App::Shutdown() { renderer_.Shutdown(); }
```

140.

```
< Functions of the application 136 > +=
void App::ReloadShaders() { shadersOk_ = renderer_.ReloadShaders(root_ / "shaders",
    ↪ shaderLog_); }
```

141. The world matrix of the solid is a rotation around the vertical axis by the angle that the animation advances. Being a pure rotation is what allows the inspector to transform normals with it directly (*Find the probe on the solid and express the normals in world space 64*)).

```
< Functions of the application 136 > +=
XMMATRIX App::World() const { return XMMatrixRotationY(rotAngle_); }
```

142. *One frame of the interface.* The order of the steps matters only in one respect: the probe must be decided (by the panels that show maps) before the viewport is drawn, because the viewport shows the probe's arrows.

```
< Functions of the application 136 > +=
void App::Frame(float dt) {
    < Advance the animations and handle the keyboard 143 >

    < Create the dock space and its initial layout 144 >

    < If there are no materials, say why and stop 145 >

    < Build the panels and run the probe 146 >
}
```

143. The animations advance by the time elapsed since the previous frame, *dt*, so their speed does not depend on the frame rate. The space bar toggles the rotation of the solid unless a text field is being edited.

```
< Advance the animations and handle the keyboard 143 > +=
if (rotate_) rotAngle_ = std::fmod(rotAngle_ + rotSpeed_ * dt, 2 * kPi);
if (light_.autoOrbit) light_.azimuth = std::fmod(light_.azimuth + light_.speed * dt, 2 *
    ↪ kPi);
if (ImGui::IsKeyPressed(ImGuiKey_F5)) ReloadShaders();
if (ImGui::IsKeyPressed(ImGuiKey_Space) && !ImGui::GetIO().WantTextInput) rotate_ =
    ↪ !rotate_;
```

This code is used in section 142.

144. The panels are *docked* into the main window. The first time the program runs, there is no saved layout, and we build one (§147); later runs restore the layout the user left, which Dear ImGui saves in `imgui.ini`. On the second frame we bring the “Scene” tab to the front of the dock it shares with “Concepts”.

```

< Create the dock space and its initial layout 144 > ≡
const ImGuiID dockId = ImGui::DockSpaceOverViewport(0, nullptr,
    ⇨ ImGuiDockNodeFlags_PassthruCentralNode);
static bool layoutBuilt = false;
if (!layoutBuilt) {
    layoutBuilt = true;
    ImGuiDockNode* node = ImGui::DockBuilderGetNode(dockId);
    if (!node || node->IsLeafNode()) BuildDockLayout(dockId);
} else if (ImGui::GetFrameCount() == 2) {
    ImGui::SetWindowFocus("Scene"); // Scene and Concepts share a dock; start on Scene
}

```

This code is used in section 142.

145.

```

< If there are no materials, say why and stop 145 > ≡
if (library_.Items().empty()) {
    ImGui::Begin("Viewport");
    ImGui::TextColored(ImVec4(1, 0.4f, 0.4f, 1), "%s", loadError_.c_str());
    ImGui::End();
    return;
}

```

This code is used in section 142.

146. This is the protocol of §125 in action. When the probe is valid, the inspector is run with the current material, shape, rotation and settings, and its answer is kept in `info_` for the viewport and, in the next frame, for the inspector panel.

```

< Build the panels and run the probe 146 > ≡
// The probe is driven by whatever map image is hovered during this frame.
hoveredThisFrame_ = false;
UiMaterials();
UiScene();
UiMaps();
UiInspector();
if (!hoveredThisFrame_ && !pinned_) probeValid_ = false;
if (probeValid_) {
    info_ = Inspect(Current(), probeUV_.x, probeUV_.y, shape_, World(), bump_);
} else {
    info_ = HoverInfo{};
}
UiViewport();
UiConcepts();

```

This code is used in section 142.

147. The initial layout splits the window into three columns: on the left the list of materials above the controls; in the middle the 3D viewport; on the right the maps above the inspector.

```

< Functions of the application 136 > + ≡
void App::BuildDockLayout(ImGuiID dockId) {
    ImGui::DockBuilderRemoveNode(dockId);
    ImGui::DockBuilderAddNode(dockId, ImGuiDockNodeFlags_DockSpace);
    ImGui::DockBuilderSetNodeSize(dockId, ImGui::GetMainViewport()->Size);
    ImGuiID center = dockId;
    ImGuiID left = ImGui::DockBuilderSplitNode(center, ImGuiDir_Left, 0.21f, nullptr,
        ⇨ &center);
    ImGuiID right = ImGui::DockBuilderSplitNode(center, ImGuiDir_Right, 0.46f, nullptr,
        ⇨ &center);
    ImGuiID leftBottom = ImGui::DockBuilderSplitNode(left, ImGuiDir_Down, 0.55f, nullptr,
        ⇨ &left);
    ImGuiID rightBottom = ImGui::DockBuilderSplitNode(right, ImGuiDir_Down, 0.66f, nullptr,
        ⇨ &right);
    ImGui::DockBuilderDockWindow("Materials", left);
    ImGui::DockBuilderDockWindow("Scene", leftBottom);
    ImGui::DockBuilderDockWindow("Concepts", leftBottom);
    ImGui::DockBuilderDockWindow("Maps", right);
    ImGui::DockBuilderDockWindow("Inspector", rightBottom);
    ImGui::DockBuilderDockWindow("Viewport", center);
    ImGui::DockBuilderFinish(dockId);
}

```

```
}

```

148. The “Materials” panel lists the materials and tells where the selected one comes from. Crediting the sources is a matter of courtesy even when, as here, the licence does not require it.

```
< Functions of the application 136 > +=
void App::UiMaterials() {
    ImGui::Begin("Materials");
    < List the materials with their thumbnails 149 >

    < Show the origin of the current material 150 >
    ImGui::End();
}

```

149. Each entry is a selectable row as tall as a thumbnail, over which the thumbnail and the two lines of text are drawn. Clicking a row selects the material (and loads it, should it not be loaded yet).

```
< List the materials with their thumbnails 149 > =
auto& items = library_.Items();
for (int i = 0; i < int(items.size()); ++i) {
    ImGui::PushID(i);
    const Material& m = items[i];
    if (ImGui::Selectable("##sel", current_ == i, 0, ImVec2(0, 36))) {
        current_ = i;
        library_.EnsureLoaded(i, renderer_.Device(), renderer_.Context());
    }
    ImGui::SameLine(0, 0);
    ImGui::SetCursorPosX(ImGui::GetStyle().WindowPadding.x);
    if (m.loaded && m.maps[MapColor])
        ImGui::Image(Tex(m.maps[MapColor]->srvLinear.Get()), ImVec2(36, 36));
    else
        ImGui::Dummy(ImVec2(36, 36));
    ImGui::SameLine();
    ImGui::BeginGroup();
    ImGui::TextUnformatted(m.name.c_str());
    ImGui::TextDisabled("%s", m.site.c_str());
    ImGui::EndGroup();
    ImGui::PopID();
}

```

This code is used in section 148.

150. The button opens the page of the material in the default web browser.

```
< Show the origin of the current material 150 > =
ImGui::SeparatorText("Origin of this material");
const Material& m = Current();
ImGui::Text("Source:  %s", m.site.c_str());
ImGui::Text("Author:  %s", m.author.c_str());
ImGui::Text("License: %s", m.license.c_str());
ImGui::TextWrapped("%s", m.url.c_str());
if (ImGui::Button("Open source page")) {
    const std::wstring url(m.url.begin(), m.url.end());
    ShellExecuteW(nullptr, L"open", url.c_str(), nullptr, nullptr, SW_SHOWNORMAL);
}
if (m.bumpFromHeight)
    ImGui::TextWrapped("Note: this pack ships no separate bump map, so the height
        ↪(displacement) "
        "map is used as the bump map.");

```

This code is used in section 148.

151. The “Scene” panel holds every control of the scene, grouped under headings.

```
< Functions of the application 136 > +=
void App::UiScene() {
    ImGui::Begin("Scene");
    < Controls of the object 152 >

    < Controls of the camera 153 >

    < Controls of the light 154 >

```

```

    < Controls of the maps and the shading 155 >

    < Controls of the shaders 156 >
    ImGui::End();
}

```

152. The rotation angle has its own slider, so that the animation can be stopped and the solid turned by hand to a chosen angle.

```

< Controls of the object 152 > ≡
ImGui::SeparatorText("Object");
int s = int(shape_);
const char* shapes[] = {ShapeName(Shape::Sphere), ShapeName(Shape::Pyramid),
    ↪ShapeName(Shape::Cylinder),
    ShapeName(Shape::Capsule)};
if (ImGui::Combo("Shape", &s, shapes, int(Shape::Count))) shape_ = Shape(s);
ImGui::Checkbox("Rotate object (Space)", &rotate_);
ImGui::SliderFloat("Rotation speed", &rotSpeed_, -2.0f, 2.0f, "%.2f rad/s");
float deg = XMConvertToDegrees(rotAngle_);
if (ImGui::SliderFloat("Rotation angle", &deg, 0, 360, "%.0f deg")) rotAngle_ =
    ↪XMConvertToRadians(deg);

```

This code is used in section 151.

153. The angles are stored in radians; `SliderAngle` shows them in degrees.

```

< Controls of the camera 153 > ≡
ImGui::SeparatorText("Camera (left-drag / wheel in viewport)");
ImGui::SliderAngle("Cam yaw", &camera_.yaw, -180, 180);
ImGui::SliderAngle("Cam pitch", &camera_.pitch, -89, 89);
ImGui::SliderFloat("Distance", &camera_.distance, 1.5f, 12.0f);

```

This code is used in section 151.

154.

```

< Controls of the light 154 > ≡
ImGui::SeparatorText("Light (right-drag in viewport)");
ImGui::Checkbox("Orbit light automatically", &light_.autoOrbit);
ImGui::SliderAngle("Light azimuth", &light_.azimuth, -180, 180);
ImGui::SliderAngle("Light elevation", &light_.elevation, -89, 89);
ImGui::SliderFloat("Light distance", &light_.radius, 1.5f, 8.0f);
ImGui::SliderFloat("Intensity", &light_.intensity, 0.0f, 10.0f);
ImGui::ColorEdit3("Light color", light_.color, ImGuiColorEditFlags_NoInputs);
ImGui::SameLine();
ImGui::SliderFloat("Orbit speed", &light_.speed, -3.0f, 3.0f);

```

This code is used in section 151.

155. These are the controls that make the experiments possible: each map can be switched off, the strength of the bump and the scale of the displacement can be exaggerated, and the view can show any intermediate quantity of the shader.

```

< Controls of the maps and the shading 155 > ≡
ImGui::SeparatorText("Maps used for shading");
ImGui::Checkbox("Color", &useColor_);
ImGui::SameLine();
ImGui::Checkbox("Bump", &useBump_);
ImGui::SameLine();
ImGui::Checkbox("Displacement", &bump_.useDisplacement);
ImGui::SameLine();
ImGui::Checkbox("Roughness", &useRough_);
ImGui::SliderFloat("Bump strength", &bump_.strength, 0.0f, 30.0f);
ImGui::SliderFloat("Displacement scale", &bump_.displacementScale, 0.0f, 0.3f);
ImGui::SliderFloat("Displacement mid-level", &bump_.displacementMid, 0.0f, 1.0f);
ImGui::BeginDisabled(useRough_);
ImGui::SliderFloat("Constant roughness", &constRough_, 0.04f, 1.0f);
ImGui::EndDisabled();
ImGui::SliderFloat("Metallic", &metallic_, 0.0f, 1.0f);
ImGui::SliderFloat("Ambient", &ambient_, 0.0f, 0.3f);
ImGui::Combo("View", &viewMode_, kViewModes, IM_ARRAYSIZE(kViewModes));

```

```
ImGui::Checkbox("Show probe arrows (N, N', T, B)", &showArrows_);
```

This code is used in section 151.

156.

```
< Controls of the shaders 156 > ≡
ImGui::SeparatorText("Shaders");
if (ImGui::Button("Reload shaders (F5)")) ReloadShaders();
ImGui::SameLine();
ImGui::TextColored(shadersOk_ ? ImVec4(0.4f, 0.9f, 0.4f, 1) : ImVec4(1, 0.4f, 0.4f, 1),
    shadersOk_ ? "OK" : "compile error");
if (!shaderLog_.empty()) ImGui::TextWrapped("%s", shaderLog_.c_str());
This code is used in section 151.
```

157. *Map images and the probe.* Every image of a map in the interface—the small tiles and the large view in the inspector—is drawn by this function, which also turns the mouse position into texture coordinates. If the image occupies the screen rectangle with top-left corner **m** and size (W, H) , the mouse at **p** points at

$$(u, v) = \left(\frac{p_x - m_x}{W}, \frac{p_y - m_y}{H} \right),$$

clamped to the unit square. Since screen y grows downwards, like v , no flip is needed: one more reason for the Direct3D convention.

```
< Functions of the application 136 > +≡
void App::MapImage(MapKind kind, const ImVec2& size) {
    const auto& map = Current().maps[kind];
    if (!map) {
        ImGui::Button("not available", size);
        return;
    }
    ImGui::Image(Tex(map->srvLinear.Get()), size);
    const ImVec2 mn = ImGui::GetItemRectMin();
    ImDrawList* dl = ImGui::GetWindowDrawList();
    < Update the probe from the mouse 158 >
    < Draw the crosshair of the probe 159 >
}
```

158. Hovering moves an unpinned probe; a left click pins it at the mouse; a right click releases it; a double click also makes this map the one shown large in the inspector.

```
< Update the probe from the mouse 158 > ≡
if (ImGui::IsItemHovered()) {
    const ImVec2 mp = ImGui::GetMousePos();
    const XMFLDAT2 uv{std::clamp((mp.x - mn.x) / size.x, 0.0f, 1.0f), std::clamp((mp.y -
        ↪mn.y) / size.y, 0.0f, 1.0f)};
    hoveredThisFrame_ = true;
    if (!pinned_) {
        probeUV_ = uv;
        probeValid_ = true;
    }
    if (ImGui::IsMouseClicked(ImGuiMouseButton_Left)) {
        probeUV_ = uv;
        probeValid_ = pinned_ = true;
    }
    if (ImGui::IsMouseClicked(ImGuiMouseButton_Right)) pinned_ = false;
    if (ImGui::IsMouseDoubleClicked(ImGuiMouseButton_Left)) inspected_ = kind;
}
```

This code is used in section 157.

159. The crosshair is drawn on *every* map image, not only on the hovered one, so that the eye can compare the colour, the height and the roughness at the same texel. A dark line under a light one keeps it visible on any image; magenta means pinned.

```
< Draw the crosshair of the probe 159 > ≡
// Crosshair at the probe position, mirrored on every map image.
if (probeValid_) {
    const ImVec2 p(mn.x + probeUV_.x * size.x, mn.y + probeUV_.y * size.y);
    const ImU32 col = pinned_ ? IM_COL32(255, 60, 200, 255) : IM_COL32(255, 255, 255, 220);
    dl->AddLine(ImVec2(p.x, mn.y), ImVec2(p.x, mn.y + size.y), IM_COL32(0, 0, 0, 120), 3);
    dl->AddLine(ImVec2(mn.x, p.y), ImVec2(mn.x + size.x, p.y), IM_COL32(0, 0, 0, 120), 3);
}
```

```

dl->AddLine(ImVec2(p.x, mn.y), ImVec2(p.x, mn.y + size.y), col);
dl->AddLine(ImVec2(mn.x, p.y), ImVec2(mn.x + size.x, p.y), col);
dl->AddCircle(p, 5, col, 0, 2);
}

```

This code is used in section 157.

160. The “Maps” panel shows the five maps as tiles, three per row, as large as the panel allows. Clicking a tile selects it for the inspector.

```

⟨ Functions of the application 136 ⟩ +=
void App::UiMaps() {
    ImGui::Begin("Maps");
    const Material& m = Current();
    const ImVec2 avail = ImGui::GetContentRegionAvail();
    const float sp = ImGui::GetStyle().ItemSpacing.y, line =
        ↪ ImGui::GetTextLineHeightWithSpacing();
    const float tile = std::max(64.0f, std::min((avail.x - 2 *
        ↪ ImGui::GetStyle().ItemSpacing.x) / 3,
        (avail.y - 4 * line - 2 * sp) / 2));

    const MapKind kinds[] = {MapColor, MapBump, MapDisplacement, MapRoughness, MapNormal};
    for (int i = 0; i < 5; ++i) {
        if (i % 3) ImGui::SameLine();
        ImGui::BeginGroup();
        ImGui::PushID(i);
        const bool sel = inspected_ == kinds[i];
        std::string label = MapName(kinds[i]);
        if (kinds[i] == MapBump && m.bumpFromHeight) label += " (from height)";
        if (sel) ImGui::PushStyleColor(ImGuiCol_Text, ImVec4(1, 0.8f, 0.3f, 1));
        ImGui::TextUnformatted(label.c_str());
        if (sel) ImGui::PopStyleColor();
        MapImage(kinds[i], ImVec2(tile, tile));
        if (ImGui::IsItemClicked() || (ImGui::IsItemHovered() &&
            ↪ ImGui::IsMouseClicked(ImGuiMouseButton_Middle)))
            inspected_ = kinds[i];
        ImGui::PopID();
        ImGui::EndGroup();
    }
    ImGui::TextDisabled("Hover a map to probe it. Left-click pins the probe, right-click
        ↪ unpins.\n"
        "The selected map is shown large in the Inspector.");
    ImGui::End();
}

```

161. The “Inspector” panel is where the program answers the student’s question. It shows one map large, so that individual texels can be aimed at, and next to it every result of ⟨ *Inspect one texel* 62 ⟩.

```

⟨ Functions of the application 136 ⟩ +=
void App::UiInspector() {
    ImGui::Begin("Inspector");
    const Material& m = Current();
    ⟨ Choose the map to inspect 162 ⟩
    ⟨ Show the inspected map, beside or above the readouts 163 ⟩

    const HoverInfo& h = info_;
    if (!h.valid) {
        ImGui::TextDisabled("Move the mouse over a map\nto inspect it.");
    } else {
        ⟨ Show the values of the maps at the probe 164 ⟩

        ⟨ Show the normals at the probe 165 ⟩
    }
    ImGui::EndGroup();
    ImGui::End();
}

```

162.

```

⟨ Choose the map to inspect 162 ⟩ ≡
if (ImGui::BeginCombo("Map", MapName(inspected_))) {

```

```

    for (int k = 0; k < MapCount; ++k)
        if (ImGui::Selectable(MapName(MapKind(k)), inspected_ == k)) inspected_ =
            ↪MapKind(k);
    ImGui::EndCombo();
}

```

This code is used in section 161.

163. The readouts need about 24 characters' width; if the panel is too narrow for the image and the readouts side by side, they are stacked.

```

⟨Show the inspected map, beside or above the readouts 163⟩≡
// Large map on the left with the readouts beside it; stacked if the panel is narrow.
const ImVec2 avail = ImGui::GetContentRegionAvail();
const float readoutWidth = 24 * ImGui::GetFontSize();
const bool sideBySide = avail.x - readoutWidth >= 200;
const float side = sideBySide ? std::max(128.0f, std::min(avail.x - readoutWidth, avail.y -
    ↪8))
    : std::max(128.0f, std::min(avail.x, avail.y * 0.38f));
MapImage(inspected_, ImVec2(side, side));
if (sideBySide) ImGui::SameLine();
ImGui::BeginGroup();

```

This code is used in section 161.

164. First the plain values: the coordinates in normalized texture space and in texels, and the value of each map.

```

⟨Show the values of the maps at the probe 164⟩≡
ImGui::Text("Probe %s", pinned_ ? "(pinned - right-click a map to release)" : "(hover)");
if (ImGui::BeginTable("vals", 3, ImGuiTableFlags_SizingFixedFit | ImGuiTableFlags_RowBg)) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn(); ImGui::TextUnformatted("u, v");
    ImGui::TableNextColumn(); ImGui::Text("%.4f, %.4f", h.uv.x, h.uv.y);
    ImGui::TableNextColumn();
    const auto& c = m.maps[MapColor];
    if (c) ImGui::Text("texel (%d, %d)", int(h.uv.x * c->width), int(h.uv.y * c->height));
    ImGui::TableNextRow();
    ImGui::TableNextColumn(); ImGui::TextUnformatted("Color (sRGB)");
    ImGui::TableNextColumn(); ImGui::Text("%.3f, %.3f, %.3f", h.color.x, h.color.y,
        ↪h.color.z);
    ImGui::TableNextColumn();
    ImGui::ColorButton("##c", ImVec4(h.color.x, h.color.y, h.color.z, 1),
        ↪ImGuiColorEditFlags_NoTooltip, ImVec2(36, 0));
    ScalarRow("Bump value", h.bump, "Height used only to tilt the shading normal.");
    ScalarRow("Displacement", h.displacement, "Height used to move vertices along the
        ↪normal.");
    ScalarRow("Spec. roughness", h.roughness, "0 = mirror-like, 1 = fully rough (GGX alpha =
        ↪r^2).");
    ImGui::EndTable();
}

```

This code is used in section 161.

165. Then the normals, in the order of the derivation of §61: the slope, the tangent-space normal, and its world-space version together with the frame that produced it.

For comparison the panel also decodes the material's own normal map, $\mathbf{n} = 2\mathbf{rgb} - 1$. The material's authors computed it from a high-resolution model with their own strength, so the numbers differ from ours; but the *direction* in which it leans agrees with the one computed from the bump map, which is a good check that our convention for **B** matches the DirectX one.

```

⟨Show the normals at the probe 165⟩≡
ImGui::SeparatorText("Normal computed from the bump map");
ImGui::Text("slope dh/dx = %.4f, dh/dy = %.4f (per texel)", h.slope.x, h.slope.y);
ImGui::TextDisabled("n_ts = normalize(-k*dh/dx, -k*dh/dy, 1), k = %.1f", bump_.strength);
if (ImGui::BeginTable("normals", 3, ImGuiTableFlags_SizingFixedFit | ImGuiTableFlags_RowBg))
    ↪{
    Vec3Row("Tangent space n_ts", h.normalTangent, true,
        "Relative to the surface: x along T (+u), y along B (+v), z along N.\nSwatch =
        ↪n*0.5+0.5.");
    if (m.maps[MapNormal]) {
        const XMFLAT4 t = m.maps[MapNormal]->Sample(h.uv.x, h.uv.y);
    }
}

```

```

XMFLOAT3 pn;
XMStoreFloat3(&pn, XMVector3Normalize(XMVectorSet(t.x * 2 - 1, t.y * 2 - 1, t.z * 2
    ↪ - 1, 0)));
Vec3Row("Pack normal map", pn, true,
    "Decoded from the material's own (DirectX convention) normal map, for
    ↪ comparison.\n"
    "It was baked with a different strength, so only the direction trend
    ↪ matches.");
}
if (h.onSurface) {
    Vec3Row("World n' (bumped)", h.normalBumped, true, "n' = T*n_ts.x + B*n_ts.y +
    ↪ N*n_ts.z, rotated to world.");
    Vec3Row("World N (geometric)", h.normalGeom, true, "Normal of the shape itself,
    ↪ without bump.");
    Vec3Row("World T (dP/du)", h.tangent, false, "Tangent: direction of increasing u.");
    Vec3Row("World B (dP/dv)", h.bitangent, false, "Bitangent: direction of increasing
    ↪ v.");
    Vec3Row("World position", h.position, false, "Displaced surface point (magenta ring
    ↪ in 3D).");
}
ImGui::EndTable();
}
if (!h.onSurface)
    ImGui::TextDisabled("This uv is not on the %s's main UV chart.", ShapeName(shape_));

```

This code is used in section 161.

166. The “Viewport” panel shows the 3D scene. Its size decides the size of the off-screen image, so the image always fills the panel exactly.

⟨ Functions of the application 136 ⟩ +≡

```

void App::UiViewport() {
    ImGui::PushStyleVar(ImGuiStyleVar_WindowPadding, ImVec2(0, 0));
    ImGui::Begin("Viewport");
    ImGui::PopStyleVar();
    const ImVec2 size = ImGui::GetContentRegionAvail();
    const UINT w = UINT(std::max(1.0f, size.x)), h = UINT(std::max(1.0f, size.y));

    ⟨ Compute the camera matrices 167 ⟩

    ⟨ Fill the frame constants 168 ⟩

    ⟨ Collect the lines of the light marker and the probe arrows 169 ⟩

    ⟨ Show the rendered image 170 ⟩

    ⟨ Orbit the camera and the light with the mouse 171 ⟩

    ⟨ Draw the legend over the image 172 ⟩
    ImGui::End();
}

```

167. The view matrix looks from the eye at the origin with y up; the projection has a vertical field of view of 45° and the aspect ratio of the panel. Both are the right-handed variants, in agreement with §5.

⟨ Compute the camera matrices 167 ⟩ ≡

```

const XMFLOAT3 eye = camera_.Eye();
const XMFLOAT3 lp = light_.Position();
const XMATRIX view = XMMatrixLookAtRH(XMLoadFloat3(&eye), XMVectorZero(), XMVectorSet(0, 1,
    ↪ 0, 0));
const XMATRIX proj = XMMatrixPerspectiveFovRH(XMConvertToRadians(45), float(w) / float(h),
    ↪ 0.05f, 100.0f);

```

This code is used in section 166.

168. Every field of the constant buffer is set here, from the state of the application. Two details: a map is used only if it is switched on *and* present; and the mipmap level for the displacement is chosen as explained in ⟨ The vertex shader: displacement mapping 69 ⟩.

⟨ Fill the frame constants 168 ⟩ ≡

```

const Material& m = Current();

```

```

FrameConstants fc{};
XMStoreFloat4x4(&fc.world, World());
XMStoreFloat4x4(&fc.viewProj, view * proj);
fc.cameraPos = eye;
fc.lightPos = lp;
fc.lightIntensity = light_.intensity;
fc.lightColor = {light_.color[0], light_.color[1], light_.color[2]};
fc.bumpStrength = bump_.strength;
fc.dispScale = bump_.displacementScale;
fc.dispMid = bump_.displacementMid;
// Sample the height at a mip whose resolution matches the mesh grid (avoids aliasing).
const auto& disp = m.maps[MapDisplacement];
fc.dispLod = disp ? std::max(0.0f, std::log2(float(disp->width) / Renderer::kMeshSegments))
    ↪: 0;
fc.metallic = metallic_;
fc.ambient = ambient_;
fc.viewMode = viewMode_;
fc.useColor = useColor_ && m.maps[MapColor];
fc.useBump = useBump_ && m.maps[MapBump];
fc.useDisp = bump_.useDisplacement && disp;
fc.useRough = useRough_ && m.maps[MapRoughness];
fc.constRoughness = constRough_;
fc.hoverUV = info_.uv;
fc.hoverOn = info_.valid ? 1.0f : 0.0f;
This code is used in section 166.

```

169. The arrows of the probe start at the (displaced) surface point. Their colours follow a tradition: red, green and blue for the axes **T**, **B** and—well, the normal gets yellow for the geometric one and cyan for the bumped one, so that the two can be told apart when they nearly coincide.

```

⟨Collect the lines of the light marker and the probe arrows 169⟩≡
std::vector<LineVertex> lines;
// Light marker: three small circles.
const XMFL0AT4 lc{1, 0.9f, 0.3f, 1};
AddCircle(lines, lp, XMVectorSet(1, 0, 0, 0), XMVectorSet(0, 1, 0, 0), 0.08f, lc);
AddCircle(lines, lp, XMVectorSet(0, 1, 0, 0), XMVectorSet(0, 0, 1, 0), 0.08f, lc);
AddCircle(lines, lp, XMVectorSet(1, 0, 0, 0), XMVectorSet(0, 0, 1, 0), 0.08f, lc);
if (showArrows_ && info_.onSurface) {
    const XMVECTOR p = XMLoadFloat3(&info_.position);
    AddArrow(lines, p, XMLoadFloat3(&info_.normalGeom), 0.45f, {1, 0.85f, 0.1f, 1}); // N
    AddArrow(lines, p, XMLoadFloat3(&info_.normalBumped), 0.55f, {0.2f, 0.95f, 1, 1}); //
    ↪N'
    AddArrow(lines, p, XMLoadFloat3(&info_.tangent), 0.25f, {1, 0.25f, 0.25f, 1}); // T
    AddArrow(lines, p, XMLoadFloat3(&info_.bitangent), 0.25f, {0.3f, 1, 0.3f, 1}); // B
}

```

This code is used in section 166.

170. If the scene could not be drawn, the panel shows the compiler's messages instead, and an invisible button takes the place of the image so that the mouse code below still works.

```

⟨Show the rendered image 170⟩≡
ID3D11ShaderResourceView* img = renderer_.RenderScene(w, h, fc, m, shape_, lines);
if (img) {
    ImGui::Image(Tex(img), ImVec2(float(w), float(h)));
} else {
    ImGui::TextColored(ImVec4(1, 0.4f, 0.4f, 1), "Shaders not available:\n%s",
    ↪shaderLog_.c_str());
    ImGui::InvisibleButton("vp", ImVec2(float(w), float(h)));
}

```

This code is used in section 166.

171. Dragging changes the angles in proportion to the mouse movement; the wheel changes the distance by a constant *factor* per notch, which feels uniform at every distance. Pitch and elevation stop just short of the poles, where the “up” vector of the view would be parallel to the direction of view.

```

⟨Orbit the camera and the light with the mouse 171⟩≡
// Mouse interaction
if (ImGui::IsItemHovered()) {
    ImGuiIO& io = ImGui::GetIO();

```

```

    if (ImGui::IsMouseDown(ImGuiMouseButton_Left, 0)) {
        camera_.yaw -= io.MouseDelta.x * 0.008f;
        camera_.pitch = std::clamp(camera_.pitch + io.MouseDelta.y * 0.008f, -1.55f, 1.55f);
    }
    if (ImGui::IsMouseDown(ImGuiMouseButton_Right, 0)) {
        light_.azimuth -= io.MouseDelta.x * 0.01f;
        light_.elevation = std::clamp(light_.elevation + io.MouseDelta.y * 0.01f, -1.55f,
        ↪1.55f);
    }
    if (io.MouseWheel != 0)
        camera_.distance = std::clamp(camera_.distance * std::pow(0.9f, io.MouseWheel),
        ↪1.5f, 12.0f);
}
camera_.yaw = std::remainder(camera_.yaw, 2 * kPi);
light_.azimuth = std::remainder(light_.azimuth, 2 * kPi);

```

This code is used in section 166.

172. A legend names what is on the screen and reminds the user of the mouse controls.

```

⟨ Draw the legend over the image 172 ⟩ ≡
// Legend overlay
const ImVec2 o = ImGui::GetItemRectMin();
ImGuiDrawList* dl = ImGui::GetWindowDrawList();
float y = o.y + 8;
auto legend = [&](ImU32 col, const char* text) {
    dl->AddRectFilled(ImVec2(o.x + 10, y + 4), ImVec2(o.x + 22, y + 10), col);
    dl->AddText(ImVec2(o.x + 28, y), IM_COL32(230, 230, 230, 255), text);
    y += 17;
};
char title[160];
snprintf(title, sizeof(title), "%s on %s | %s", m.name.c_str(), ShapeName(shape_),
        ↪kViewModes[viewMode_]);
dl->AddText(ImVec2(o.x + 10, y), IM_COL32(255, 255, 255, 255), title);
y += 20;
if (showArrows_ && info_.onSurface) {
    legend(IM_COL32(255, 217, 25, 255), "N geometric normal");
    legend(IM_COL32(51, 242, 255, 255), "N' normal from bump map");
    legend(IM_COL32(255, 64, 64, 255), "T tangent (+u)");
    legend(IM_COL32(77, 255, 77, 255), "B bitangent (+v)");
}
legend(IM_COL32(255, 230, 77, 255), "light");
dl->AddText(ImVec2(o.x + 10, o.y + size.y - 22), IM_COL32(180, 180, 180, 255),
    "Left-drag: orbit camera Right-drag: orbit light Wheel: zoom Space: rotate
    ↪object");

```

This code is used in section 166.

173. The “Concepts” panel contains, in a few paragraphs, the theory that this document develops at length. Each paragraph ends with something to try in the program.

```

⟨ Functions of the application 136 ⟩ +≡
void App::UiConcepts() {
    ImGui::Begin("Concepts");
    auto section = [](const char* title, const char* body) {
        if (ImGui::CollapsingHeader(title)) {
            ImGui::PushTextWrapPos(0);
            ImGui::TextUnformatted(body);
            ImGui::PopTextWrapPos();
        }
    };
    ⟨ Explanations of the concepts 174 ⟩
    ImGui::End();
}

```

174.

```

⟨ Explanations of the concepts 174 ⟩ ≡
section("UV space",
    "Every vertex stores a texture coordinate (u,v). Normalized texture space runs from
    ↪(0,0) at the "

```

```

    "top-left texel corner to (1,1) at the bottom-right (Direct3D convention: v grows
    ↪downwards). "
    "The rasterizer interpolates uv across each triangle and the pixel shader samples
    ↪the textures "
    "there. Choose the 'UV coordinates' view to see the mapping; notice how the sphere
    ↪stretches "
    "u around its equator and pinches it at the poles, and how the pyramid gives each
    ↪face a quarter "
    "of the texture.");
section("Bump mapping",
    "A bump map is a height field h(u,v). It does NOT move geometry: the shader only
    ↪tilts the "
    "normal according to the slope of h. With central differences, dh/dx =
    ↪(h(x+1)-h(x-1))/2, and the "
    "surface  $z = k \cdot h$  has tangent-space normal  $n_{ts} = \text{normalize}(-k \cdot dh/dx, -k \cdot dh/dy, 1)$ .
    ↪Flat areas give "
    "(0,0,1). Turn displacement off and look at the silhouette: it stays perfectly
    ↪smooth.");
section("Tangent space and TBN",
    "Tangent space is a frame attached to the surface: T points to +u, B to +v and N is
    ↪the geometric "
    "normal. The matrix [T B N] converts a tangent-space normal to object/world space: "
    " $n' = T \cdot n.x + B \cdot n.y + N \cdot n.z$ . That is why the same bump map works on any shape. The
    ↪Inspector "
    "shows  $n_{ts}$  and the resulting world-space  $n'$  for the probed texel; the cyan arrow is
    ↪ $n'$ .");
section("Normal maps",
    "A normal map stores  $n_{ts}$  directly as a color ( $rgb = n \cdot 0.5 + 0.5$ ), precomputed from a
    ↪height map or "
    "a high-poly model. Mostly bluish because flat areas are (0,0,1). Conventions differ
    ↪in the sign of "
    "green (OpenGL vs DirectX). The 'Pack normal map' row in the Inspector decodes the
    ↪material's own "
    "map so you can compare it with the one computed from the bump map.");
section("Displacement mapping",
    "Displacement really moves the surface: each vertex is pushed along its normal by "
    " $scale \cdot (h - mid)$ . It changes silhouettes and self-occlusion, but needs dense
    ↪geometry (here a "
    "256x256 grid per patch). The shading normal does not change unless a bump/normal
    ↪map is also used, "
    "which is why both are usually combined. Note the cracks on the pyramid edges and
    ↪cylinder rims: "
    "vertices shared by faces with different normals are pushed in different
    ↪directions.");
section("Specular roughness",
    "The roughness map controls the microfacet distribution of the GGX/Cook-Torrance
    ↪BRDF "
    " $(\alpha = r^2)$ . Low values give small, sharp highlights; high values spread the
    ↪highlight "
    "into a dull sheen. Orbit the light and compare shiny and rough regions of the same
    ↪material.");

```

This code is used in section 173.

175. The main program.

What remains is the frame around the picture: a Win32 window, the connection between the window and Dear ImGui, and the loop that produces one frame after another until the window is closed.

```

< File src/main.cpp 175 >≡
// Texture Explorer - an interactive Direct3D 11 tool for learning about texturing:
// color, bump, displacement and roughness maps, UV space and tangent-space normals.
#include "App.h"
#include <imgui.h>
#include <imgui_impl_dx11.h>
#include <imgui_impl_win32.h>
#include <windows.h>
#include <chrono>

extern ImGui_ImplAPI LRESULT ImGui_ImplWin32_WndProcHandler(HWND, UINT, WPARAM, LPARAM);

< Global variables of the main program 176 >

< The window procedure 177 >

< The main program 178 >

```

This code is written to the file `src/main.cpp`.

176. The window procedure is called by Windows, outside the flow of our program, so the little it has to communicate is left in global variables. A resize is not handled immediately but recorded, and applied at a quiet moment of the main loop.

```

< Global variables of the main program 176 >≡
static App* g_app = nullptr;
static UINT g_resizeW = 0, g_resizeH = 0;

```

This code is used in section 175.

177. Messages go first to Dear ImGui, which needs the mouse and the keyboard; the rest are the classic minimum of a Windows program. Swallowing `SC_KEYMENU` prevents the Alt key from freezing the program in the (non-existent) window menu.

```

< The window procedure 177 >≡
static LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    if (ImGui_ImplWin32_WndProcHandler(hwnd, msg, wParam, lParam)) return true;
    switch (msg) {
        case WM_SIZE:
            if (wParam != SIZE_MINIMIZED) {
                g_resizeW = LOWORD(lParam);
                g_resizeH = HIWORD(lParam);
            }
            return 0;
        case WM_SYSCOMMAND:
            if ((wParam & 0xffff) == SC_KEYMENU) return 0; // no ALT application menu
            break;
        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;
    }
    return DefWindowProcW(hwnd, msg, wParam, lParam);
}

```

This code is used in section 175.

178. The program begins, as Windows programs with a graphical interface do, in `wWinMain`.

```

< The main program 178 >≡
int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE, PWSTR, int) {
    < Create the window 179 >

    < Initialize the application 180 >

    < Initialize Dear ImGui 181 >

    < Run the main loop 183 >

```

```
    < Clean up 188 >
}
```

This code is used in section 175.

179. The program declares itself aware of the screen's DPI, so that Windows does not enlarge it blurrily on high-resolution displays; instead it scales its own window and, below, its fonts and widgets by the factor dpi.

```
< Create the window 179 > ≡
ImGui_ImplWin32_EnableDpiAwareness();
const float dpi = ImGui_ImplWin32_GetDpiScaleForMonitor(MonitorFromPoint(POINT{0, 0},
    ↪ MONITOR_DEFAULTTOPRIMARY));

WNDCLASSEXW wc{sizeof(wc), CS_CLASSDC, WndProc, 0, 0, hInstance, LoadIcon(nullptr,
    ↪ IDI_APPLICATION),
    LoadCursor(nullptr, IDC_ARROW), nullptr, nullptr, L"TextureExplorer",
    ↪ nullptr};
RegisterClassExW(&wc);
HWND hwnd = CreateWindowW(wc.lpszClassName, L"Texture Explorer - bump, displacement &
    ↪ roughness maps",
    WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, int(1600 *
    ↪ dpi), int(950 * dpi),
    nullptr, nullptr, hInstance, nullptr);
```

This code is used in section 178.

180. The only fatal error of the program is the absence of a Direct3D 11 device.

```
< Initialize the application 180 > ≡
App app;
g_app = &app;
if (!app.Init(hwnd)) {
    MessageBoxW(hwnd, L"Could not create a Direct3D 11 device.", L"Texture Explorer",
    ↪ MB_ICONERROR);
    return 1;
}
ShowWindow(hwnd, SW_SHOWDEFAULT);
UpdateWindow(hwnd);
```

This code is used in section 178.

181. Dear ImGui is configured with docking and keyboard navigation. It saves the layout of the panels in `imgui.ini`; we place that file next to the executable, because the current directory depends on how the program was started.

```
< Initialize Dear ImGui 181 > ≡
ImGui_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_DockingEnable | ImGuiConfigFlags_NavEnableKeyboard;
// Keep the window layout next to the executable, not in the current directory.
static std::string iniPath;
{
    char exe[MAX_PATH];
    GetModuleFileNameA(nullptr, exe, MAX_PATH);
    iniPath = exe;
    iniPath = iniPath.substr(0, iniPath.find_last_of("\\") + 1) + "imgui.ini";
    io.IniFilename = iniPath.c_str();
}
< Choose the style and the font 182 >
ImGui_ImplWin32_Init(hwnd);
ImGui_ImplDX11_Init(app.GetRenderer().Device(), app.GetRenderer().Context());
```

This code is used in section 178.

182. The dark style with slightly rounded corners, and the Windows interface font, Segoe UI, when it is available; otherwise Dear ImGui's built-in bitmap font, magnified.

```
< Choose the style and the font 182 > ≡
ImGui::StyleColorsDark();
ImGuiStyle& style = ImGui::GetStyle();
style.WindowRounding = 4;
style.FrameRounding = 3;
style.ScaleAllSizes(dpi);
```

```
const char* uiFont = "C:\\Windows\\Fonts\\segoeui.ttf";
if (GetFileAttributesA(uiFont) == INVALID_FILE_ATTRIBUTES ||
    !io.Fonts->AddFontFromFileTTF(uiFont, 17.0f * dpi))
    io.FontGlobalScale = dpi; // fall back to the built-in font
```

This code is used in section 181.

183. *The main loop.* Each iteration handles the pending messages, measures the time, builds the interface (which, as we saw, also renders the 3D scene off-screen), and finally draws the interface into the window and presents it.

```
< Run the main loop 183 > ≡
auto last = std::chrono::steady_clock::now();
bool running = true;
while (running) {
    < Process the pending window messages 184 >

    < Measure the time since the last frame 185 >

    < Build the interface and the scene 186 >

    < Draw the interface and present it 187 >
}
```

This code is used in section 178.

184. While the window is minimized there is nothing to draw, and the program sleeps instead of spinning. A pending resize is applied here, between frames, when no view of the back buffer is in use.

```
< Process the pending window messages 184 > ≡
MSG msg;
while (PeekMessageW(&msg, nullptr, 0, 0, PM_REMOVE)) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
    if (msg.message == WM_QUIT) running = false;
}
if (!running) break;
if (IsIconic(hwnd)) {
    Sleep(10);
    continue;
}
if (g_resizeW && g_resizeH) {
    app.GetRenderer().Resize(g_resizeW, g_resizeH);
    g_resizeW = g_resizeH = 0;
}
```

This code is used in section 183.

185.

```
< Measure the time since the last frame 185 > ≡
const auto now = std::chrono::steady_clock::now();
const float dt = std::chrono::duration<float>(now - last).count();
last = now;
```

This code is used in section 183.

186.

```
< Build the interface and the scene 186 > ≡
ImGui_ImplDX11_NewFrame();
ImGui_ImplWin32_NewFrame();
ImGui::NewFrame();
app.Frame(dt);
ImGui::Render();
```

This code is used in section 183.

187.

```
< Draw the interface and present it 187 > ≡
const float clear[4] = {0.08f, 0.08f, 0.09f, 1};
app.GetRenderer().BeginFrame(clear);
ImGui_ImplDX11_RenderDrawData(ImGui::GetDrawData());
app.GetRenderer().Present(true);
```

This code is used in section 183.

188. Everything is released in the reverse order of creation.

```
< Clean up 188 > ≡  
ImGui_ImplDX11_Shutdown();  
ImGui_ImplWin32_Shutdown();  
ImGui::DestroyContext();  
app.Shutdown();  
DestroyWindow(hwnd);  
UnregisterClassW(wc.lpszClassName, wc.hInstance);  
return 0;
```

This code is used in section 178.

189. Building the program.

The build is described for CMake, which generates a Visual Studio solution. Three libraries are downloaded by CMake itself the first time the project is configured, each pinned to a precise version so that the build is reproducible: Dear ImGui (the *docking* branch, which provides the dockable panels), `stb_image` and `nlohmann/json`.

```
<File CMakeLists.txt 189>≡
cmake_minimum_required(VERSION 3.20)
project(TextureExplorer LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

<Fetch the libraries 190>

<Build Dear ImGui as a static library 191>

<Build the program 192>
```

This code is written to the file `CMakeLists.txt`.

190.

```
<Fetch the libraries 190>≡
include(FetchContent)
FetchContent_Declare(imgui
    GIT_REPOSITORY https://github.com/ocornut/imgui.git
    GIT_TAG         v1.91.9b-docking
    GIT_SHALLOW    TRUE)
FetchContent_Declare(stb
    GIT_REPOSITORY https://github.com/nothings/stb.git
    GIT_TAG         2c980bb59875b0d32144a71867fbdebb2f77cd20)
FetchContent_Declare(json
    URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz)
FetchContent_MakeAvailable(imgui stb json)
```

This code is used in section 189.

191. Dear ImGui does not come with a build description of its own; it expects its few source files to be compiled with the program, together with the two “back ends” that connect it to Win32 and to Direct3D 11.

```
<Build Dear ImGui as a static library 191>≡
add_library(imgui STATIC
    ${imgui_SOURCE_DIR}/imgui.cpp
    ${imgui_SOURCE_DIR}/imgui_draw.cpp
    ${imgui_SOURCE_DIR}/imgui_tables.cpp
    ${imgui_SOURCE_DIR}/imgui_widgets.cpp
    ${imgui_SOURCE_DIR}/imgui_demo.cpp
    ${imgui_SOURCE_DIR}/backends/imgui_impl_win32.cpp
    ${imgui_SOURCE_DIR}/backends/imgui_impl_dx11.cpp)
target_include_directories(imgui PUBLIC ${imgui_SOURCE_DIR} ${imgui_SOURCE_DIR}/backends)
```

This code is used in section 189.

192. The program is a WIN32 executable (it has no console). The definition `TEXEX_ROOT` records the source folder for `FindRoot` in *<Private helpers of the application 127>*; `NOMINMAX` keeps `windows.h` from defining macros called `min` and `max`, which would break `std::min` and `std::max`. The compiler is asked for warning level 4 and strict conformance; the program compiles without a single warning.

```
<Build the program 192>≡
add_executable(TextureExplorer WIN32
    src/main.cpp
    src/App.cpp
    src/Renderer.cpp
    src/Material.cpp
    src/Mesh.cpp
    src/Inspector.cpp)
target_include_directories(TextureExplorer PRIVATE src ${stb_SOURCE_DIR})
target_link_libraries(TextureExplorer PRIVATE imgui nlohmann_json::nlohmann_json
    d3d11 dxgi d3dcompiler shell32)
target_compile_definitions(TextureExplorer PRIVATE
    TEXEX_ROOT="${CMAKE_SOURCE_DIR}" UNICODE _UNICODE NOMINMAX WIN32_LEAN_AND_MEAN)
if(MSVC)
```

```
    target_compile_options(TextureExplorer PRIVATE /W4 /permissive- /utf-8)
    set_property(DIRECTORY PROPERTY VS_STARTUP_PROJECT TextureExplorer)
endif()
```

This code is used in section [189](#).

193. Exercises.

A literate program invites modification. Here are some changes that a student can make, in increasing order of difficulty, each of which illuminates one of the ideas above.

1. In *⟨ Patches of the sphere 17 ⟩*, remove the minus sign of the z coordinate (and of the derivatives). Look at the texture, and then at the inspector's $\mathbf{T}$ and $\mathbf{B}$: what happened to the rule $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$, and to the sign w ?
2. In *⟨ The normal of a height field, on the GPU 70 ⟩* and *⟨ The normal of a height field 61 ⟩*, replace the central differences by forward differences, $h(x+1) - h(x)$. Compare the probe's normal before and after on a sharp edge of a brick.
3. Make the pyramid repeat the whole texture on each face. What must change in *⟨ Patches of the pyramid 22 ⟩*, and why can the inspector no longer find a unique point?
4. Add a view mode that shows the *difference* between the normal computed from the bump map and the decoded normal map of the material.
5. Recompute the normal in the vertex shader from the displacement map, so that displacement alone produces correct shading. (Hint: the same central differences, on the displacement map, at the level `dispLod`.)
6. Replace the vertex displacement by hardware tessellation with hull and domain shaders, so that the mesh can be coarse far from the camera.

Names of the modules.

<File CMakeLists.txt 189> Written to a file.
<File shaders/gizmo.hlsl 82> Written to a file.
<File shaders/pbr.hlsl 65> Written to a file.
<File src/App.cpp 126> Written to a file.
<File src/App.h 117> Written to a file.
<File src/Inspector.cpp 60> Written to a file.
<File src/Inspector.h 56> Written to a file.
<File src/main.cpp 175> Written to a file.
<File src/Material.cpp 37> Written to a file.
<File src/Material.h 32> Written to a file.
<File src/Mesh.cpp 14> Written to a file.
<File src/Mesh.h 7> Written to a file.
<File src/Renderer.cpp 91> Written to a file.
<File src/Renderer.h 83> Written to a file.
<File tools/fetch_textures.ps1 50> Written to a file.
<Add the base of the pyramid 24> Used in section 22.
<Add the diffuse term, the light and the ambient term 80> Used in section 77.
<Add the four sides of the pyramid 23> Used in section 22.
<Advance the animations and handle the keyboard 143> Used in section 142.
<Bilinear sampling on the CPU 41> Used in section 37.
<Build Dear ImGui as a static library 191> Used in section 189.
<Build the interface and the scene 186> Used in section 183.
<Build the panels and run the probe 146> Used in section 142.
<Build the program 192> Used in section 189.
<Bump the normal 74> Used in section 72.
<Caps of the cylinder 19> Used in section 18.
<Choose a debug view 76> Used in section 72.
<Choose the map to inspect 162> Used in section 161.
<Choose the style and the font 182> Used in section 181.
<Choose the ten materials 52> Used in section 50.
<Clean up 188> Used in section 178.
<Clear the scene target and set the pipeline state 112> Used in section 111.
<Collect the lines of the light marker and the probe arrows 169> Used in section 166.
<Compute the camera matrices 167> Used in section 166.
<Constants of a frame 66> Used in section 65.
<Constants of a frame, as seen from C++ 84> Used in section 83.
<Controls of the camera 153> Used in section 151.
<Controls of the light 154> Used in section 151.
<Controls of the maps and the shading 155> Used in section 151.
<Controls of the object 152> Used in section 151.
<Controls of the shaders 156> Used in section 151.
<Create a grey texture for missing maps 100> Used in section 92.
<Create a texture with a full mipmap chain 45> Used in section 42.
<Create the blend state for lines 97> Used in section 92.
<Create the constant buffer 98> Used in section 92.
<Create the depth buffer 109> Used in section 106.
<Create the depth states 96> Used in section 92.
<Create the dock space and its initial layout 144> Used in section 142.
<Create the line buffer 99> Used in section 92.
<Create the linear and sRGB views 46> Used in section 42.
<Create the multisampled colour target 107> Used in section 106.
<Create the rasterizer state 95> Used in section 92.
<Create the resolved image 108> Used in section 106.
<Create the sampler 94> Used in section 92.

< *Create the shader objects, keeping the old ones on failure* 105 > Used in section 103.
 < *Create the swap chain* 93 > Used in section 92.
 < *Create the window* 179 > Used in section 178.
 < *Decode the image file* 43 > Used in section 42.
 < *Describe the two vertex layouts* 104 > Used in section 103.
 < *Description and parameters of the download script* 51 > Used in section 50.
 < *Device and swap chain* 88 > Used in section 83.
 < *Directions for the lighting* 78 > Used in section 77.
 < *Download the ambientCG materials* 53 > Used in section 50.
 < *Download the Poly Haven materials* 54 > Used in section 50.
 < *Draw the crosshair of the probe* 159 > Used in section 157.
 < *Draw the interface and present it* 187 > Used in section 183.
 < *Draw the legend over the image* 172 > Used in section 166.
 < *Draw the lines* 115 > Used in section 111.
 < *Draw the surface* 114 > Used in section 111.
 < *Emit the triangles of one patch* 30 > Used in section 28.
 < *Emit the vertices of one patch* 29 > Used in section 28.
 < *Evaluate the Cook-Torrance BRDF* 77 > Used in section 72.
 < *Explanations of the concepts* 174 > Used in section 173.
 < *Fetch the libraries* 190 > Used in section 189.
 < *Fill the frame constants* 168 > Used in section 166.
 < *Find the point of the capsule's profile at arc length s* 21 > Used in section 20.
 < *Find the probe on the solid and express the normals in world space* 64 > Used in section 62.
 < *Functions of the application* 136 > Used in section 126.
 < *Functions of the renderer* 92 > Used in section 91.
 < *Global variables of the main program* 176 > Used in section 175.
 < *Helper that builds a surface point* 16 > Used in section 14.
 < *If there are no materials, say why and stop* 145 > Used in section 142.
 < *Initialize Dear ImGui* 181 > Used in section 178.
 < *Initialize the application* 180 > Used in section 178.
 < *Inspect one texel* 62 > Used in section 60.
 < *Instantiate the image decoder* 38 > Used in section 37.
 < *Interface of the inspector* 59 > Used in section 56.
 < *Interface of the mesh module* 13 > Used in section 7.
 < *Keep a CPU copy of the pixels* 44 > Used in section 42.
 < *Kinds of maps* 33 > Used in section 32.
 < *List the materials with their thumbnails* 149 > Used in section 148.
 < *Load one image into a texture* 42 > Used in section 37.
 < *Load the maps of a material on first use* 49 > Used in section 37.
 < *Map images* 34 > Used in section 32.
 < *Mark the probed texel* 81 > Used in section 72.
 < *Materials* 35 > Used in section 32.
 < *Measure the time since the last frame* 185 > Used in section 183.
 < *Mesh data* 12 > Used in section 7.
 < *Names of the maps* 39 > Used in section 37.
 < *Orbit the camera and the light with the mouse* 171 > Used in section 166.
 < *Parse the entries of the manifest* 48 > Used in section 47.
 < *Patches* 11 > Used in section 7.
 < *Patches of the capsule* 20 > Used in section 14.
 < *Patches of the cylinder* 18 > Used in section 14.
 < *Patches of the pyramid* 22 > Used in section 14.
 < *Patches of the sphere* 17 > Used in section 14.
 < *Pipeline objects* 90 > Used in section 83.
 < *Private helpers of the application* 127 > Used in section 126.
 < *Private helpers of the renderer* 87 > Used in section 83.
 < *Private member functions of the application* 121 > Used in section 117.

- ⟨ *Process the pending window messages 184* ⟩ Used in section 183.
- ⟨ *Public functions of the mesh module 25* ⟩ Used in section 14.
- ⟨ *Public interface of the application 120* ⟩ Used in section 117.
- ⟨ *Public interface of the renderer 86* ⟩ Used in section 83.
- ⟨ *Read the colour and the roughness 75* ⟩ Used in section 72.
- ⟨ *Read the manifest 47* ⟩ Used in section 37.
- ⟨ *Rebuild the tangent frame in the pixel shader 73* ⟩ Used in section 72.
- ⟨ *Reminder of the texture-space convention 15* ⟩ Used in section 14.
- ⟨ *Resources of the application 122* ⟩ Used in section 117.
- ⟨ *Run the main loop 183* ⟩ Used in section 178.
- ⟨ *Sample every map at the probe 63* ⟩ Used in section 62.
- ⟨ *Settings of bump and displacement 57* ⟩ Used in section 56.
- ⟨ *Show the inspected map, beside or above the readouts 163* ⟩ Used in section 161.
- ⟨ *Show the normals at the probe 165* ⟩ Used in section 161.
- ⟨ *Show the origin of the current material 150* ⟩ Used in section 148.
- ⟨ *Show the rendered image 170* ⟩ Used in section 166.
- ⟨ *Show the values of the maps at the probe 164* ⟩ Used in section 161.
- ⟨ *State of the inspector 125* ⟩ Used in section 117.
- ⟨ *State of the scene 123* ⟩ Used in section 117.
- ⟨ *State of the shading 124* ⟩ Used in section 117.
- ⟨ *Surface points 9* ⟩ Used in section 7.
- ⟨ *Texel access with wrap-around 40* ⟩ Used in section 37.
- ⟨ *Textures and sampler 67* ⟩ Used in section 65.
- ⟨ *The four shapes 8* ⟩ Used in section 7.
- ⟨ *The main program 178* ⟩ Used in section 175.
- ⟨ *The material library 36* ⟩ Used in section 32.
- ⟨ *The normal of a height field 61* ⟩ Used in section 60.
- ⟨ *The normal of a height field, on the GPU 70* ⟩ Used in section 65.
- ⟨ *The off-screen scene target 89* ⟩ Used in section 83.
- ⟨ *The orbiting camera 118* ⟩ Used in section 117.
- ⟨ *The orbiting light 119* ⟩ Used in section 117.
- ⟨ *The pixel shader 72* ⟩ Used in section 65.
- ⟨ *The three factors of the specular term 79* ⟩ Used in section 77.
- ⟨ *The vertex shader: displacement mapping 69* ⟩ Used in section 65.
- ⟨ *The window procedure 177* ⟩ Used in section 175.
- ⟨ *Unbind the textures and resolve the multisampled image 116* ⟩ Used in section 111.
- ⟨ *Undo the display encoding of debug colours 71* ⟩ Used in section 65.
- ⟨ *Update the probe from the mouse 158* ⟩ Used in section 157.
- ⟨ *Upload the frame constants 113* ⟩ Used in section 111.
- ⟨ *Vertex formats of the surface shader 68* ⟩ Used in section 65.
- ⟨ *Vertex layout 10* ⟩ Used in section 7.
- ⟨ *Vertices of lines 85* ⟩ Used in section 83.
- ⟨ *What the probe reports 58* ⟩ Used in section 56.
- ⟨ *Write the manifest 55* ⟩ Used in section 50.